

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

GIA BẢO ANH
NGUYỄN THỊ NHƯ HUỲNH

KHÓA LUẬN TỐT NGHIỆP
XÂY DỰNG MẠNG XÃ HỘI THÔNG MINH:
TÍCH HỢP AI KIỂM DUYỆT VÀ
CÁ NHÂN HÓA NỘI DUNG

**BUILDING A SMART SOCIAL NETWORK:
INTEGRATING AI FOR CONTENT MODERATION
AND PERSONALIZATION**

CỬ NHÂN NGÀNH KỸ THUẬT PHẦN MỀM

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

GIA BẢO ANH – 22520044

NGUYỄN THỊ NHƯ HUỖNH – 22520592

KHÓA LUẬN TỐT NGHIỆP
XÂY DỰNG MẠNG XÃ HỘI THÔNG MINH:
TÍCH HỢP AI KIỂM DUYỆT VÀ
CÁ NHÂN HÓA NỘI DUNG

**BUILDING A SMART SOCIAL NETWORK:
INTEGRATING AI FOR CONTENT MODERATION
AND PERSONALIZATION**

CỬ NHÂN NGÀNH KỸ THUẬT PHẦN MỀM

GIẢNG VIÊN HƯỚNG DẪN

THS. TRẦN THỊ HỒNG YẾN

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp được thành lập theo Quyết định số:

.....

Ngày/...../..... của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn, cô ThS. Trần Thị Hồng Yến thuộc Khoa Công nghệ Phần mềm, Trường Đại học Công nghệ Thông tin – Đại học Quốc gia TP.HCM, đã dành thời gian hướng dẫn, xem xét và đóng góp ý kiến cho đề tài “Xây dựng mạng xã hội thông minh: Tích hợp AI kiểm duyệt và cá nhân hóa nội dung”.

Đề tài được thực hiện với mục tiêu phát triển một nền tảng mạng xã hội đáp ứng nhu cầu kết nối, giao tiếp và chia sẻ thông tin. Trong quá trình thực hiện, nhóm đã nghiên cứu và phân tích yêu cầu, thiết kế giao diện, phát triển các chức năng mạng xã hội, xây dựng cơ chế cá nhân hóa nội dung, giải thích thuật ngữ văn hóa mạng, nhắn tin, thông báo và hoàn thiện hệ thống.

Mặc dù nhóm đã nỗ lực hoàn thành đề tài, báo cáo khó tránh khỏi những hạn chế do phạm vi kiến thức và thời gian thực hiện. Nhóm rất mong nhận được những ý kiến đóng góp quý báu của Quý Thầy/Cô để tiếp tục hoàn thiện sản phẩm và nâng cao kiến thức chuyên môn.

Xin chân thành cảm ơn Quý Thầy/Cô!

Nhóm sinh viên thực hiện:

Gia Bảo Anh

Nguyễn Thị Như Huỳnh

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	iv
TÓM TẮT KHÓA LUẬN	1
1 GIỚI THIỆU ĐỀ TÀI	2
1.1 Lý do chọn đề tài	2
1.2 Mục tiêu đề tài	3
1.3 Đối tượng và phạm vi nghiên cứu	3
1.3.1 Đối tượng nghiên cứu:	3
1.3.2 Phạm vi nghiên cứu:	4
1.4 Công cụ và công nghệ sử dụng	4
1.5 Phương pháp thực hiện	5
2 CƠ SỞ LÝ THUYẾT	7
2.1 Công nghệ nền tảng	7
2.1.1 TypeScript	7
2.1.2 React Native	8
2.1.3 Next.js	8
2.1.4 Node.js	9
2.2 Cơ sở dữ liệu	10
2.2.1 MongoDB	10
2.3 Dịch vụ hỗ trợ	11
2.3.1 Pusher	11
2.3.2 Cloudinary	11
2.3.3 Socket.io	12
2.4 Kiểm duyệt nội dung	13
2.4.1 Biểu thức chính quy và đối sánh từ khóa (Rule-based Filter) .	13

2.4.2	Mô hình ngôn ngữ lớn – LLM (Gemini API)	13
2.4.3	Prompt Engineering	14
2.4.4	Kiến trúc pipeline 2 tầng và Human-in-the-loop	14
2.5	Giải thích thuật ngữ và ngữ cảnh văn hóa mạng	15
2.5.1	Đối sánh từ điển dựa trên biểu thức chính quy	15
2.5.2	Chiến lược Batch API Call	15
2.6	Cá nhân hóa nội dung	16
2.6.1	Content-Based Filtering	16
2.6.2	Popularity-Based Filtering	16
2.6.3	Công thức tính điểm xếp hạng (Time Decay, CTR, Impression Tracking)	16
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	18
3.1	Khảo sát hiện trạng	18
3.1.1	Các vấn đề của mạng xã hội hiện tại	18
3.1.2	Khảo sát các nền tảng hiện có	19
3.2	Danh sách các yêu cầu	20
3.2.1	Yêu cầu chức năng	20
3.2.2	Yêu cầu phi chức năng	25
3.2.3	Yêu cầu chất lượng	27
3.3	Kiến trúc hệ thống	29
3.3.1	Kiến trúc tổng thể	29
3.3.2	Module AI Content Moderation	31
3.3.3	Module Personalized Feed	34
3.3.4	Module giải thích thuật ngữ và ngữ cảnh văn hóa mạng	38
3.4	Thiết kế Use Case	40
3.4.1	Danh sách các Actor	40
3.4.2	Danh sách các Use Case chính	42
3.4.3	Đặc tả chi tiết Use Case	45
3.5	Sơ đồ Activity	59

3.6	Thiết kế dữ liệu	65
3.6.1	Sơ đồ lớp (Class Diagram)	65
3.6.2	Sơ đồ ERD	65
3.6.3	Mô tả chi tiết các collection	65
4	XÂY DỰNG ỨNG DỤNG	66
4.1	Giao diện ứng dụng di động	66
4.2	Giao diện trang quản trị	85
4.3	Minh họa các module hỗ trợ thông minh	86
4.4	Liên kết mã nguồn	87
	TÀI LIỆU THAM KHẢO	88

DANH MỤC HÌNH

2.1	TypeScript	7
2.2	React Native	8
2.3	Next.js	9
2.4	Node.js	9
2.5	MongoDB	10
2.6	Pusher	11
2.7	Cloudinary	12
2.8	Socket.io	13
2.9	Gemini	14
3.1	Kiến trúc tổng thể của hệ thống	30
3.2	Luồng xử lý của 2-Tier Moderation Pipeline	34
3.3	Luồng xử lý của Personalized Feed	38
3.4	Luồng giải thích thuật ngữ và ngữ cảnh văn hóa mạng	40
3.5	Sơ đồ activity đăng ký tài khoản và xác thực OTP	60
3.6	Sơ đồ activity đăng nhập và xác thực hai yếu tố	61
3.7	Sơ đồ activity tạo bài viết và xử lý hậu kỳ bằng AI	62
3.8	Sơ đồ activity truy xuất feed cá nhân hóa	63
3.9	Sơ đồ activity báo cáo vi phạm và xử lý quản trị	64
4.1	Màn hình Đăng nhập	68
4.2	Màn hình Đăng ký	69
4.3	Màn hình Xác thực tài khoản	70
4.4	Màn hình Xác thực hai yếu tố	71
4.5	Màn hình Feed Khám phá	72
4.6	Màn hình Tạo bài viết	73
4.7	Màn hình Chi tiết bài viết	74

4.8	Màn hình Tìm kiếm	75
4.9	Màn hình Hồ sơ cá nhân	76
4.10	Màn hình Quan hệ người dùng	77
4.11	Màn hình Bài viết đã lưu	78
4.12	Màn hình Thông báo	79
4.13	Màn hình Danh sách trò chuyện	80
4.14	Màn hình Chi tiết trò chuyện	81
4.15	Màn hình Báo cáo nội dung	82
4.16	Màn hình Quên mật khẩu	83
4.17	Màn hình Đổi mật khẩu	84
4.18	Màn hình Giải thích thuật ngữ văn hóa mạng	85

DANH MỤC BẢNG

3.1	Nhóm Quản lý người dùng	20
3.2	Nhóm Quản lý bài đăng và tương tác	21
3.3	Nhóm Module AI Kiểm duyệt nội dung	22
3.4	Nhóm Module AI Cá nhân hóa Feed	23
3.5	Nhóm Module giải thích thuật ngữ và ngữ cảnh văn hóa mạng	24
3.6	Nhóm Nhấn tin và Thông báo	25
3.7	Yêu cầu phi chức năng	25
3.8	Yêu cầu chất lượng	27
3.9	Trọng số Scoring Engine theo chế độ người dùng	35
3.10	Trọng số và hệ số tín hiệu theo loại tương tác	36
3.9	Danh sách các actor của hệ thống	40
3.10	Danh sách các use case chính	42

DANH MỤC TỪ VIẾT TẮT

STT	Ký hiệu	Tiếng Anh	Tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	API	Application Programming Interface	Giao diện lập trình ứng dụng
3	UI	User Interface	Giao diện người dùng
4	UX	User Experience	Trải nghiệm người dùng
5	2FA	Two-Factor Authentication	Xác thực hai yếu tố
6	LLM	Large Language Model	Mô hình ngôn ngữ lớn
7	CTR	Click-Through Rate	Tỉ lệ nhấp
8	NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
9	MVP	Minimum Viable Product	Sản phẩm khả dụng tối thiểu
10	REST	Representational State Transfer	Kiểu kiến trúc truyền tải trạng thái đại diện
11	ERD	Entity Relationship Diagram	Sơ đồ thực thể quan hệ
12	SSR	Server-Side Rendering	Kết xuất phía máy chủ
13	SSG	Static Site Generation	Tạo trang tĩnh
14	CDN	Content Delivery Network	Mạng phân phối nội dung
15	JSON	JavaScript Object Notation	Định dạng dữ liệu
16	BSON	Binary JavaScript Object Notation	Định dạng JSON nhị phân
17	HITL	Human-in-the-loop	Con người trong vòng lặp quyết định

TÓM TẮT KHÓA LUẬN

Trước thực trạng các nền tảng mạng xã hội hiện nay còn đối mặt với nội dung độc hại lan tràn và trải nghiệm người dùng chưa được cá nhân hóa hiệu quả, đề tài hướng đến xây dựng một ứng dụng mạng xã hội hiện đại tích hợp trí tuệ nhân tạo, tập trung vào ba tính năng cốt lõi: kiểm duyệt nội dung tự động, đề xuất nội dung cá nhân hóa và giải thích thuật ngữ, ngữ cảnh văn hóa mạng.

Chương 1: Giới thiệu đề tài – trình bày lý do chọn đề tài, mục tiêu, đối tượng, phạm vi nghiên cứu, công cụ công nghệ được áp dụng và phương pháp thực hiện phát triển hệ thống.

Chương 2: Cơ sở lý thuyết – giới thiệu các công nghệ nền tảng (TypeScript, Next.js, React Native, MongoDB, Pusher, Cloudinary) và các mô hình, kỹ thuật được áp dụng trong hệ thống (Regex, LLM – Gemini API, Content-Based Filtering, Time Decay, CTR, Impression Tracking).

Chương 3: Phân tích và thiết kế hệ thống – xác định các yêu cầu nghiệp vụ, hệ thống và chất lượng, từ đó thiết kế Use-Case, sơ đồ Activity, sơ đồ tuần tự, sơ đồ lớp và ERD nhằm mô tả chi tiết kiến trúc và luồng xử lý của các chức năng chính.

Chương 4: Xây dựng ứng dụng – trình bày quá trình hiện thực hóa ứng dụng di động và trang quản trị web, đồng thời mô tả cách tích hợp các tính năng AI (kiểm duyệt nội dung, cá nhân hóa, giải thích thuật ngữ và ngữ cảnh văn hóa mạng) vào luồng xử lý thực tế của hệ thống.

Kết luận – tổng kết các nhóm chức năng và ba module hỗ trợ thông minh đã được hiện thực; đồng thời trình bày các hạn chế, phạm vi đánh giá hiện tại và định hướng phát triển trong tương lai.

Chương 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày lý do chọn đề tài, mục tiêu, đối tượng, phạm vi nghiên cứu, công cụ công nghệ được áp dụng và phương pháp thực hiện phát triển hệ thống.

1.1. Lý do chọn đề tài

Hiện nay, sự bùng nổ của internet và smartphone đã thúc đẩy nhu cầu kết nối, giao tiếp và chia sẻ thông tin trực tuyến ngày càng tăng cao. Mạng xã hội không chỉ giúp mọi người kết nối với bạn bè, gia đình và cộng đồng từ khắp nơi trên thế giới, mà còn trở thành kênh cung cấp thông tin, giải trí và quảng bá doanh nghiệp quan trọng trong đời sống hiện đại.

Tuy nhiên, các nền tảng mạng xã hội hiện nay vẫn tồn tại nhiều thách thức chưa được giải quyết triệt để. Nội dung độc hại như ngôn từ thù ghét, bắt nạt trực tuyến và spam vẫn lan tràn do việc kiểm duyệt thủ công tốn kém và chậm trễ, gây ảnh hưởng tiêu cực đến trải nghiệm người dùng, đặc biệt với các nhóm dễ bị tổn thương. Bên cạnh đó, người dùng thường bị "ngập" trong lượng thông tin khổng lồ mà không tìm được nội dung phù hợp với sở thích và nhu cầu của mình, do các thuật toán đề xuất truyền thống còn đơn giản và thiếu cá nhân hóa.

Xuất phát từ thực tế đó, nhóm quyết định thực hiện đề tài **"Xây dựng mạng xã hội thông minh: tích hợp AI kiểm duyệt và cá nhân hóa nội dung"**. Ứng dụng không chỉ cung cấp đầy đủ các chức năng mạng xã hội cốt lõi (đăng bài, tương tác, nhắn tin) mà còn tích hợp AI để tự động kiểm duyệt nội dung vi phạm, cá nhân hóa trải nghiệm người dùng và hỗ trợ giải thích thuật ngữ, ngữ cảnh văn hóa mạng, hướng đến một môi trường giao tiếp trực tuyến an toàn, thông minh và thân thiện hơn.

1.2. Mục tiêu đề tài

Đề tài hướng đến xây dựng một ứng dụng mạng xã hội di động hoàn chỉnh, tích hợp trí tuệ nhân tạo nhằm tạo ra môi trường giao tiếp trực tuyến an toàn, thông minh và phù hợp với nhu cầu từng cá nhân. Cụ thể, đề tài đặt ra các mục tiêu sau:

Xây dựng nền tảng mạng xã hội đầy đủ tính năng cốt lõi: Hệ thống hỗ trợ xác thực đa dạng (email, số điện thoại, Google) kết hợp bảo mật hai yếu tố (2FA); đăng bài đa phương tiện (văn bản, hình ảnh, video); tương tác xã hội (like, comment, share, save); quản lý kết nối (follow/unfollow, bestfriend, chặn người dùng); nhắn tin và gọi điện thời gian thực hỗ trợ cả chat 1-1 lẫn nhóm.

Triển khai hệ thống AI kiểm duyệt nội dung: Xây dựng pipeline kiểm duyệt 2 tầng kết hợp rule-based filter và mô hình AI (Gemini API) nhằm tự động phát hiện và xử lý nội dung độc hại, ngôn từ thù ghét và spam với độ chính xác mục tiêu đạt $\geq 85\%$.

Phát triển hệ thống đề xuất nội dung cá nhân hóa: Xây dựng thuật toán gợi ý dựa trên lịch sử tương tác và sở thích người dùng, kết hợp Content-Based Filtering và Popularity-Based nhằm nâng cao mức độ gắn kết và sự hài lòng của người dùng.

Tích hợp tính năng giải thích thuật ngữ văn hóa mạng: Tự động phát hiện và giải thích các thuật ngữ slang, meme, cultural reference trong bài viết thông qua LLM, giúp thu hẹp khoảng cách thế hệ và văn hóa giữa các nhóm người dùng.

Xây dựng trang quản trị cho Admin: Cung cấp dashboard giám sát người dùng, xét duyệt nội dung vi phạm, thống kê hoạt động hệ thống và theo dõi hiệu suất các module AI.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu:

Đề tài tập trung nghiên cứu các vấn đề và đối tượng sau:

- Các mô hình, thuật toán AI ứng dụng trong kiểm duyệt nội dung độc hại (rule-based filter, LLM – Gemini API) và đề xuất nội dung cá nhân hóa (Content-

Based Filtering, Popularity-Based).

- Kỹ thuật đối sánh từ điển dựa trên biểu thức chính quy (Regex) phục vụ việc phát hiện thuật ngữ văn hóa mạng và xác định vị trí của thuật ngữ trong văn bản.
- Người dùng cuối có nhu cầu kết nối, chia sẻ và giao tiếp trực tuyến, đặc biệt là giới trẻ tại Việt Nam.
- Quản trị viên (Admin) có nhu cầu giám sát và kiểm duyệt nội dung trên nền tảng.

1.3.2. Phạm vi nghiên cứu:

- **Nền tảng:** Ứng dụng di động (iOS/Android) và trang quản trị web (Web Admin).
- **Ngôn ngữ hỗ trợ:** Tiếng Việt và tiếng Anh.
- **Quy mô triển khai:** Môi trường demo với 100–500 người dùng thử nghiệm.
- **Phạm vi kiểm duyệt:** Tập trung vào nội dung văn bản và hình ảnh. Đối với video, phiên bản hiện tại chỉ kiểm tra ảnh thu nhỏ (thumbnail) khi dữ liệu này được cung cấp.
- **Phạm vi AI:** Sử dụng các API và mô hình AI có sẵn (Gemini API) thay vì tự huấn luyện mô hình từ đầu, phù hợp với giai đoạn MVP.
- **Ngoài phạm vi:** Không xử lý video streaming, live stream và các tính năng mạng xã hội nâng cao như marketplace, AR/VR.

1.4. Công cụ và công nghệ sử dụng

Thiết kế UI/UX: Figma – thiết kế wireframe và prototype giao diện người dùng.

Ngôn ngữ lập trình: TypeScript – sử dụng xuyên suốt cho cả mobile, web admin và backend; Node.js – xây dựng server-side và REST API.

Nền tảng phát triển:

- React Native – xây dựng ứng dụng di động đa nền tảng (iOS/Android).
- Next.js – xây dựng trang quản trị web (Web Admin).

Cơ sở dữ liệu: MongoDB – lưu trữ dữ liệu người dùng, bài viết, tin nhắn và kết quả phân tích AI.

Dịch vụ hỗ trợ:

- Pusher – xử lý thông báo và tin nhắn thời gian thực.
- Cloudinary – lưu trữ và quản lý tài nguyên đa phương tiện (hình ảnh, video).

AI & API:

- Gemini API (Google) – kiểm duyệt nội dung độc hại và giải thích thuật ngữ, ngữ cảnh văn hóa mạng.
- Socket.io – hỗ trợ giao tiếp realtime giữa client và server.

Quản lý dự án & mã nguồn:

- GitHub – quản lý phiên bản mã nguồn.
- Google Sheet – theo dõi tiến độ và phân chia công việc.

1.5. Phương pháp thực hiện

Đề tài được thực hiện theo quy trình phát triển phần mềm có hệ thống, kết hợp nghiên cứu lý thuyết và triển khai thực tiễn, cụ thể như sau:

Thu thập và phân tích yêu cầu: Khảo sát các ứng dụng mạng xã hội hiện có trên thị trường (Facebook, Instagram, TikTok) để xác định các tính năng cốt lõi và điểm còn hạn chế. Từ đó xác định danh sách yêu cầu chức năng và phi chức năng cho hệ thống.

Nghiên cứu công nghệ: Tìm hiểu các công nghệ nền tảng (React Native, Next.js, Node.js, MongoDB) và nghiên cứu các mô hình, thuật toán AI phù hợp cho

từng tính năng, bao gồm Gemini API cho kiểm duyệt nội dung và giải thích thuật ngữ văn hóa mạng, Content-Based Filtering cho hệ thống gợi ý.

Thiết kế hệ thống: Xây dựng bộ tài liệu thiết kế gồm sơ đồ Use-Case, sơ đồ Activity, sơ đồ tuần tự, sơ đồ lớp và ERD. Thiết kế kiến trúc tích hợp AI vào luồng xử lý của hệ thống. Thiết kế giao diện người dùng bằng Figma trước khi tiến hành lập trình.

Phát triển và tích hợp: Phát triển song song frontend (React Native cho mobile, Next.js cho web admin) và backend (Node.js). Tích hợp các module AI vào luồng xử lý thực tế, đảm bảo tối ưu chi phí gọi API và độ trễ hệ thống.

Kiểm thử và đánh giá: Kiểm thử toàn bộ tính năng, trong đó đánh giá độ chính xác của module AI kiểm duyệt nội dung theo các chỉ số Precision, Recall và F1-Score; đánh giá hệ thống gợi ý qua CTR và thời gian tương tác; kiểm thử bảo mật và hiệu năng hệ thống.

Hoàn thiện và triển khai: Sửa lỗi, tối ưu hóa hiệu năng, thu thập feedback từ nhóm người dùng thử nghiệm nhỏ hoặc dữ liệu mô phỏng và hoàn thiện tài liệu báo cáo khóa luận.

Chương 2. CƠ SỞ LÝ THUYẾT

Chương này giới thiệu hệ sinh thái công nghệ nền tảng (Web/Mobile/Database) và các mô hình, kỹ thuật được áp dụng để xử lý dữ liệu, kiểm duyệt nội dung và tối ưu hóa trải nghiệm cá nhân hóa trong hệ thống.

2.1. Công nghệ nền tảng

2.1.1. TypeScript

TypeScript là ngôn ngữ lập trình mã nguồn mở được Microsoft phát triển, là tập mở rộng của JavaScript với khả năng kiểm tra kiểu dữ liệu tĩnh (static typing). Thay vì phát hiện lỗi ở thời điểm chạy như JavaScript thuần, TypeScript cho phép phát hiện lỗi ngay trong quá trình viết code, giúp giảm thiểu bug và tăng khả năng bảo trì mã nguồn. Trong đề tài này, TypeScript được sử dụng xuyên suốt cho cả ứng dụng di động, trang quản trị web và backend, đảm bảo tính nhất quán và dễ mở rộng của toàn bộ hệ thống.

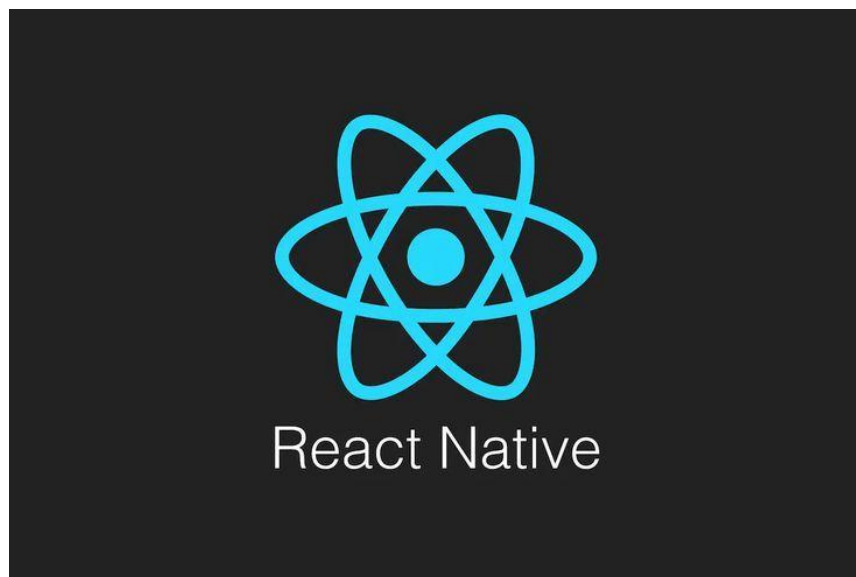


Hình 2.1: TypeScript

Nguồn: <https://i1-e.pinimg.com/736x/8c/0a/b3/8c0ab3baec0ebc6cfbad22cbe206d0cd.jpg>

2.1.2. React Native

React Native là framework phát triển ứng dụng di động đa nền tảng do Meta (Facebook) xây dựng, cho phép viết một bộ mã nguồn duy nhất và triển khai đồng thời trên cả iOS lẫn Android. React Native sử dụng các component giao diện native thật sự thay vì WebView, giúp ứng dụng có hiệu năng và trải nghiệm người dùng gần với ứng dụng native. Trong đề tài, React Native được chọn để xây dựng ứng dụng di động nhằm tiết kiệm thời gian phát triển mà vẫn đảm bảo chất lượng giao diện trên cả hai nền tảng.



Hình 2.2: React Native

Nguồn: <https://i1-e.pinimg.com/736x/56/ee/5e/56ee5ed8b4954396d11465c77966fd7c.jpg>

2.1.3. Next.js

Next.js là framework React do Vercel phát triển, hỗ trợ Server-Side Rendering (SSR) và Static Site Generation (SSG), giúp tối ưu hiệu năng và SEO so với React thuần. Next.js cung cấp hệ thống định tuyến dựa trên cấu trúc thư mục, tích hợp sẵn API Routes và nhiều tính năng tối ưu hóa hiện đại. Trong đề tài, Next.js được sử dụng để xây dựng trang quản trị web (Web Admin), nơi admin có thể giám sát người dùng, xét duyệt nội dung vi phạm và theo dõi hiệu suất các module AI.

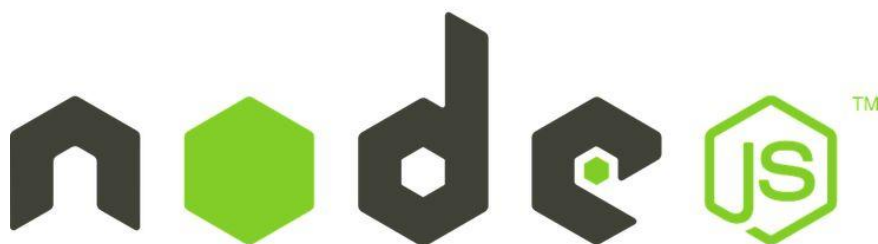


Hình 2.3: Next.js

Nguồn: <https://i1-e.pinimg.com/736x/5d/91/65/5d91653dadcd207fb95ec2ddcd746ebe2.jpg>

2.1.4. Node.js

Node.js là môi trường chạy JavaScript phía server, được xây dựng trên engine V8 của Chrome. Node.js hoạt động theo mô hình bất đồng bộ (asynchronous) và hướng sự kiện (event-driven), cho phép xử lý đồng thời nhiều kết nối với tài nguyên thấp, phù hợp với các ứng dụng yêu cầu realtime và tải cao. Trong đề tài, Node.js đảm nhận vai trò xây dựng backend, xử lý REST API và điều phối các luồng gọi AI.



Hình 2.4: Node.js

Nguồn: <https://i1-e.pinimg.com/1200x/a7/17/ea/a717eafa290bf333c4dd1c86076c5b9e.jpg>

2.2. Cơ sở dữ liệu

2.2.1. MongoDB

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL hướng tài liệu (document-oriented), lưu trữ dữ liệu dưới dạng JSON/BSON thay vì bảng quan hệ như SQL truyền thống. Điểm mạnh của MongoDB là schema linh hoạt, cho phép lưu trữ các cấu trúc dữ liệu phức tạp và không đồng nhất mà không cần định nghĩa trước. Khả năng mở rộng theo chiều ngang (horizontal scaling) cũng giúp MongoDB phù hợp với các hệ thống có lượng dữ liệu tăng trưởng nhanh. Trong đề tài, MongoDB được sử dụng để lưu trữ toàn bộ dữ liệu của hệ thống bao gồm thông tin người dùng, bài viết, tin nhắn, lịch sử tương tác và kết quả phân tích AI (bao gồm vị trí ký tự của các thuật ngữ văn hóa mạng được phát hiện).



Hình 2.5: MongoDB

Nguồn: <https://i1-e.pinimg.com/736x/e8/ab/b4/e8abb49f605c7f61553c8268ef0a56aa.jpg>

2.3. Dịch vụ hỗ trợ

2.3.1. Pusher

Pusher là dịch vụ đám mây chuyên cung cấp hạ tầng giao tiếp thời gian thực thông qua giao thức WebSocket. Pusher cho phép server đẩy dữ liệu xuống client ngay lập tức mà không cần client liên tục gửi request kiểm tra, giúp giảm tải đáng kể cho hệ thống. Trong đề tài, Pusher được sử dụng để xử lý các thông báo đẩy (push notification) và cập nhật trạng thái thời gian thực như thông báo like, comment, follow cho người dùng.



Hình 2.6: Pusher

Nguồn: <https://i1-e.pinimg.com/1200x/ad/46/8d/ad468d0ae4a44bcfe90db8ec42bf07fe.jpg>

2.3.2. Cloudinary

Cloudinary là nền tảng quản lý tài nguyên đa phương tiện trên đám mây, hỗ trợ lưu trữ, xử lý và phân phối hình ảnh, video với tốc độ cao thông qua CDN (Content Delivery Network) toàn cầu. Cloudinary cung cấp các tính năng xử lý ảnh tự động như nén, thay đổi kích thước và chuyển đổi định dạng ngay tại thời điểm phân phối. Trong đề tài, Cloudinary đảm nhận vai trò lưu trữ và quản lý toàn bộ tài nguyên đa

phương tiện do người dùng đăng tải, giúp giảm tải cho server và đảm bảo tốc độ tải nội dung.



Hình 2.7: Cloudinary

Nguồn: <https://i1-e.pinimg.com/736x/83/fd/93/83fd9301ea17bb2af6b92e841b282201.jpg>

2.3.3. Socket.io

Socket.io là thư viện JavaScript hỗ trợ giao tiếp hai chiều (bidirectional) theo thời gian thực giữa client và server, được xây dựng trên nền WebSocket với khả năng tự động fallback sang các phương thức khác khi WebSocket không khả dụng. Socket.io hỗ trợ các tính năng như phòng (rooms), namespace và xác nhận sự kiện (event acknowledgement), phù hợp cho các ứng dụng chat và realtime. Trong đề tài, Socket.io được sử dụng để xây dựng hệ thống nhắn tin thời gian thực hỗ trợ chat 1-1 và nhóm.



Hình 2.8: Socket.io

Nguồn: <https://i1-e.pinimg.com/1200x/38/a1/2f/38a12f0863bfe75c163af8493f90aff0.jpg>

2.4. Kiểm duyệt nội dung

2.4.1. Biểu thức chính quy và đối sánh từ khóa (Rule-based Filter)

Biểu thức chính quy (Regular Expression – Regex) cho phép mô tả và phát hiện các mẫu ký tự trong văn bản. Trong hệ thống, kỹ thuật này được kết hợp với danh sách từ khóa để nhận diện từ ngữ không phù hợp, cấu trúc câu có dấu hiệu thù ghét và các tín hiệu spam như ký tự lặp, nhiều liên kết hoặc tỉ lệ chữ hoa bất thường. Văn bản được chuẩn hóa trước khi đối sánh nhằm giảm ảnh hưởng của khác biệt chữ hoa, chữ thường và dấu tiếng Việt. Cách tiếp cận này có chi phí xử lý thấp, phù hợp làm tầng sàng lọc ban đầu trước khi cân nhắc gọi mô hình ngôn ngữ.

2.4.2. Mô hình ngôn ngữ lớn – LLM (Gemini API)

Mô hình ngôn ngữ lớn (Large Language Model – LLM) là các mô hình AI được huấn luyện trên tập dữ liệu văn bản khổng lồ với hàng tỷ tham số, có khả năng hiểu và sinh ngôn ngữ tự nhiên ở mức độ sâu. Gemini là họ mô hình LLM do Google DeepMind phát triển, được cung cấp thông qua Gemini API với khả năng xử lý đa phương thức (text, image). Trong đề tài, Gemini API được tích hợp vào tầng thứ hai của pipeline kiểm duyệt để phân tích ngữ nghĩa sâu các nội dung có dấu hiệu vi phạm, trả về các chỉ số định lượng như `toxic_score`, `hate_speech_score` và `spam_score` phục

vụ cho việc phân loại và xử lý tự động.



Hình 2.9: Gemini

Nguồn: <https://i1-e.pinimg.com/1200x/96/82/6c/96826cc153b5f2b00a8bb3ef0dedbdea.jpg>

2.4.3. Prompt Engineering

Prompt Engineering là kỹ thuật thiết kế và tối ưu hóa câu lệnh đầu vào (prompt) nhằm định hướng LLM trả về kết quả chính xác, nhất quán và đúng định dạng mong muốn. Một prompt được thiết kế tốt cần xác định rõ vai trò của mô hình, định nghĩa cụ thể các tiêu chí đánh giá, yêu cầu định dạng đầu ra (thường là JSON) và xử lý các trường hợp ngoại lệ. Trong đề tài, Prompt Engineering được sử dụng để tăng tính nhất quán của cấu trúc JSON đầu ra. Hệ thống vẫn kiểm tra và xử lý các trường hợp phản hồi sai định dạng hoặc lỗi dịch vụ nhằm tránh làm gián đoạn luồng đăng bài của người dùng.

2.4.4. Kiến trúc pipeline 2 tầng và Human-in-the-loop

Pipeline kiểm duyệt 2 tầng là chiến lược xử lý kết hợp rule-based filter ở tầng 1 và AI moderation ở tầng 2, nhằm cân bằng tốc độ xử lý, chi phí gọi API và khả năng phân tích ngữ cảnh. Tầng 1 sàng lọc nội dung với chi phí thấp; các trường hợp nghi vấn, nội dung bị báo cáo, nội dung từ tài khoản mới hoặc mẫu kiểm tra ngẫu nhiên có thể được chuyển lên tầng 2. Human-in-the-loop (HITL) là cơ chế đưa con người vào

vòng lặp quyết định cho các trường hợp AI chưa đủ tự tin, cụ thể là các nội dung có điểm vi phạm nằm trong ngưỡng trung gian (0.5–0.8). Cơ chế này giúp hạn chế sai sót do False Positive (chặn nhầm nội dung sạch) và False Negative (bỏ sót nội dung vi phạm), đảm bảo chất lượng kiểm duyệt trong thực tế.

2.5. Giải thích thuật ngữ và ngữ cảnh văn hóa mạng

2.5.1. Đối sánh từ điển dựa trên biểu thức chính quy

Hệ thống lưu các thuật ngữ văn hóa mạng cùng mẫu biểu thức chính quy trong MongoDB. Khi phân tích bài viết, từng mẫu đang hoạt động được đối sánh với nội dung để phát hiện thuật ngữ và xác định vị trí ký tự (character offset). Kết quả này được sử dụng để gắn phần giải thích do Gemini cung cấp và hỗ trợ client làm nổi bật đúng đoạn văn bản. Cách tiếp cận hiện tại là đối sánh từ điển dựa trên Regex, không phải mô hình nhận dạng thực thể có tên.

2.5.2. Chiến lược Batch API Call

Batch API Call là chiến lược gộp nhiều yêu cầu xử lý vào một lần gọi API duy nhất thay vì gọi riêng lẻ từng yêu cầu, nhằm giảm số lượng request, tiết kiệm chi phí và giảm độ trễ tổng thể. Trong đề tài, sau khi rule-based engine phát hiện toàn bộ thuật ngữ văn hóa mạng trong một bài viết, tất cả thuật ngữ được gộp vào một lần gọi Gemini API duy nhất để lấy toàn bộ giải thích (nghĩa, nguồn gốc, tone cảm xúc, ghi chú ngữ cảnh). Kết quả được lưu trực tiếp vào document MongoDB kèm vị trí ký tự, giúp frontend hiển thị tooltip mà không cần gọi lại Gemini API khi người dùng tương tác.

2.6. Cá nhân hóa nội dung

2.6.1. Content-Based Filtering

Content-Based Filtering là phương pháp lọc và gợi ý nội dung dựa trên đặc trưng của chính nội dung đó và lịch sử tương tác của người dùng, thay vì so sánh với hành vi của người dùng khác. Hệ thống phân tích các đặc trưng của bài viết như chủ đề, hashtag, từ khóa và so sánh với hồ sơ sở thích của từng người dùng được xây dựng từ lịch sử like, comment, share và thời gian xem. Ưu điểm của phương pháp này là không phụ thuộc vào dữ liệu người dùng khác, phù hợp với giai đoạn đầu khi hệ thống chưa có đủ dữ liệu tương tác.

2.6.2. Popularity-Based Filtering

Popularity-Based Filtering là phương pháp gợi ý nội dung dựa trên mức độ phổ biến chung của bài viết, thường được tính theo số lượt like, comment, share và độ mới của bài viết. Phương pháp này được sử dụng song song với Content-Based Filtering để giải quyết vấn đề Cold Start – tức là khi người dùng mới chưa có đủ lịch sử tương tác để xây dựng hồ sơ sở thích. Sự kết hợp hai phương pháp giúp hệ thống luôn có nội dung để gợi ý trong mọi trường hợp.

2.6.3. Công thức tính điểm xếp hạng (Time Decay, CTR, Impression Tracking)

Để xếp hạng và sắp xếp thứ tự hiển thị bài viết trong feed, hệ thống sử dụng công thức tính điểm kết hợp nhiều yếu tố:

Time Decay là cơ chế giảm điểm của bài viết theo thời gian, đảm bảo nội dung mới luôn được ưu tiên hiển thị hơn nội dung cũ dù có lượng tương tác cao hơn. Điểm time decay thường được tính theo hàm mũ hoặc hàm logarit dựa trên khoảng thời gian kể từ khi bài viết được đăng.

CTR (Click-Through Rate) là tỉ lệ giữa số lần người dùng nhấn vào bài viết so với số lần bài viết được hiển thị, phản ánh mức độ thu hút của nội dung. CTR cao cho thấy bài viết đang được gợi ý đúng đối tượng quan tâm.

Impression Tracking là cơ chế theo dõi số lần một bài viết được hiển thị đến người dùng, kết hợp với thời gian dừng lại xem (dwell time) để đánh giá mức độ phù hợp thực sự của nội dung được gợi ý. Dữ liệu này được phản hồi liên tục vào hệ thống để cải thiện chất lượng gợi ý theo thời gian.

Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này khảo sát hiện trạng các nền tảng mạng xã hội hiện có, xác định các yêu cầu chức năng, phi chức năng và chất lượng của hệ thống. Trên cơ sở đó, kiến trúc tổng thể cùng ba module AI cốt lõi được thiết kế, kết hợp với các sơ đồ Use Case, Activity và thiết kế cơ sở dữ liệu nhằm mô tả đầy đủ luồng xử lý và cấu trúc dữ liệu của hệ thống.

3.1. Khảo sát hiện trạng

3.1.1. Các vấn đề của mạng xã hội hiện tại

Mạng xã hội đã trở thành một phần không thể thiếu trong đời sống hiện đại. Tính đến tháng 4/2026, có khoảng 5,79 tỷ tài khoản người dùng mạng xã hội trên toàn thế giới, tức hơn 2 trong 3 người trên Trái Đất đang sử dụng mạng xã hội mỗi tháng [1], [2]. Tuy nhiên, sự phát triển bùng nổ này cũng kéo theo những thách thức nghiêm trọng mà các nền tảng hiện tại vẫn chưa giải quyết triệt để.

Vấn đề nội dung độc hại và kiểm duyệt

Sự lan tràn của nội dung độc hại bao gồm ngôn từ thù ghét (hate speech), bắt nạt trực tuyến và spam là thách thức cấp bách hàng đầu. Dù các nền tảng lớn đã đầu tư mạnh vào hệ thống kiểm duyệt tự động, kết quả vẫn còn nhiều hạn chế. Trong quý 1/2024, Meta đã gỡ bỏ hoặc gắn cờ 16 triệu nội dung chứa hate speech trên Facebook và Instagram, tuy nhiên có đến 148.000 nội dung bị gỡ nhầm và sau đó phải được khôi phục lại [3], [4]. Đáng lo ngại hơn, nghiên cứu từ USC Viterbi Information Sciences Institute ghi nhận mức tăng 50% về hate speech, 30% về nội dung kỳ thị đồng tính và 42% về nội dung phân biệt chủng tộc trên X (Twitter) từ tháng 1/2022 đến tháng 6/2023 sau khi nền tảng này cắt giảm kiểm duyệt [5]. Ở góc độ kỹ thuật, các công cụ NLP kiểm duyệt tự động thường mang thiên kiến thuật toán, dễ phân loại sai nội dung khi người dùng sử dụng ngôn ngữ đặc thù của các nhóm văn hóa khác nhau, đặc biệt

với tiếng Việt và các ngôn ngữ ít tài nguyên [6], [7].

Vấn đề cá nhân hóa nội dung và ”bong bóng lọc”

Các thuật toán đề xuất nội dung hiện nay, dù mang lại trải nghiệm cá nhân hóa, lại vô tình tạo ra hiện tượng ”filter bubble”. Một tổng quan hệ thống gồm 30 nghiên cứu trong giai đoạn 2015–2025 cho thấy các hệ thống thuật toán có xu hướng khuếch đại tính đồng nhất về tư tưởng, hạn chế sự đa dạng quan điểm, và đặc biệt ảnh hưởng đến nhóm người dùng trẻ với nhận thức về thuật toán còn hạn chế [8]. Hệ thống đề xuất content-based filtering chỉ dựa trên lịch sử tương tác có thể dẫn đến tình trạng ”overspecialization”, khiến người dùng bị mắc kẹt trong vòng lặp nội dung hẹp và không khám phá được các chủ đề mới [9].

Vấn đề khoảng cách thế hệ và văn hóa

Ngôn ngữ mạng (slang, meme, cultural reference) phát triển nhanh chóng tạo ra rào cản giao tiếp giữa các thế hệ và nhóm văn hóa. Các nền tảng hiện tại chưa có cơ chế hỗ trợ người dùng hiểu ngữ cảnh văn hóa của nội dung, dẫn đến hiểu nhầm, xung đột không đáng có và giảm khả năng tiếp cận của nhóm người dùng lớn tuổi hoặc đến từ nền văn hóa khác.

3.1.2. Khảo sát các nền tảng hiện có

Nhóm tiến hành phân tích ba nền tảng mạng xã hội phổ biến nhất hiện nay để xác định hướng phát triển phù hợp.

Facebook

Facebook hiện là nền tảng mạng xã hội lớn nhất thế giới với 3,22 tỷ người dùng hoạt động hàng tháng tính đến đầu năm 2026 [1]. Facebook sở hữu hệ thống kiểm duyệt nội dung quy mô lớn và đầy đủ tính năng mạng xã hội. Tuy nhiên, từ tháng 1/2025, Meta thông báo cắt giảm kiểm duyệt chủ động (proactive enforcement) đối với một số loại nội dung độc hại, bao gồm hate speech, và chuyển sang mô hình gắn nhãn cộng đồng tương tự X [3], [10]. Điều này dẫn đến tổng số nội dung hate speech bị gỡ bỏ giảm từ 7,4 triệu xuống còn 3,4 triệu trong quý 1/2025 so với cùng kỳ năm 2024, làm dấy lên lo ngại về chất lượng môi trường nội dung trên nền tảng này [3].

TikTok

TikTok nổi bật với hệ thống đề xuất nội dung cá nhân hóa mạnh mẽ dựa trên hành vi người dùng theo thời gian thực. Tuy nhiên, thuật toán của TikTok bị đánh giá là có xu hướng thu hẹp sự tập trung của người dùng nhanh hơn so với YouTube, tạo ra hiệu ứng "echo chamber" mạnh hơn [9], và do tốc độ tiêu thụ nội dung nhanh, thông tin sai lệch có thể lan truyền rộng trước khi các hệ thống kiểm duyệt kịp phản ứng. Ngoài ra, nền tảng này còn thiếu các tính năng hỗ trợ giao tiếp đa thể hệ và giải thích ngữ cảnh văn hóa.

Twitter/X

Twitter/X cung cấp không gian trao đổi thông tin nhanh và rộng nhưng đang đối mặt với bài toán kiểm duyệt nghiêm trọng sau khi cắt giảm đội ngũ moderation từ năm 2022 [5]. Ngoài ra, X đã hạn chế đáng kể quyền truy cập API cho nghiên cứu học thuật, tạo ra rào cản lớn cho việc phát triển các giải pháp kiểm duyệt và phân tích nội dung từ bên ngoài.

Nhận xét chung

Qua khảo sát, có thể thấy các nền tảng hiện có đều chưa giải quyết đồng thời ba vấn đề: kiểm duyệt nội dung chính xác và nhất quán, cá nhân hóa nội dung cân bằng tránh filter bubble, và hỗ trợ giao tiếp đa văn hóa thể hệ. Đây chính là khoảng trống mà hệ thống trong đề tài này hướng đến giải quyết.

3.2. Danh sách các yêu cầu

3.2.1. Yêu cầu chức năng

Hệ thống được tổ chức thành sáu nhóm chức năng chính như sau.

Quản lý người dùng Bảng 3.1: Nhóm Quản lý người dùng

Mã	Tên chức năng	Mô tả
FR-01	Đăng ký tài khoản	Người dùng đăng ký bằng email, xác thực qua mã OTP

FR-02	Đăng nhập / Đăng xuất	Xác thực bằng email/mật khẩu, cấp JWT access token và refresh token
FR-03	Quản lý hồ sơ cá nhân	Cập nhật ảnh đại diện, tên hiển thị, bio, sở thích chủ đề
FR-04	Theo dõi / Hủy theo dõi	Người dùng follow/unfollow người dùng khác, cập nhật social graph
FR-05	Tìm kiếm người dùng và bài đăng	Tìm kiếm theo tên, username, hashtag, nội dung
FR-06	Chặn người dùng	Người dùng có thể chặn tài khoản khác, ẩn toàn bộ nội dung của họ

Quản lý bài đăng và tương tác Bảng 3.2: Nhóm Quản lý bài đăng và tương tác

Mã	Tên chức năng	Mô tả
FR-07	Tạo bài đăng	Đăng nội dung văn bản, hình ảnh; gắn hashtag, mention; hệ thống tự động kích hoạt pipeline kiểm duyệt và giải thích thuật ngữ văn hóa mạng
FR-08	Chỉnh sửa bài đăng	Tác giả chỉnh sửa bài; hệ thống tự động kích hoạt lại pipeline kiểm duyệt và giải thích thuật ngữ văn hóa mạng
FR-09	Xóa bài đăng	Tác giả hoặc Admin xóa bài đăng
FR-10	Bình luận	Người dùng bình luận, trả lời bình luận (nested comment)
FR-11	Tương tác bài đăng	Like, share, save bài đăng
FR-12	Báo cáo nội dung	Người dùng báo cáo bài đăng hoặc bình luận vi phạm để quản trị viên xem xét

Module AI Kiểm duyệt nội dung Bảng 3.3: Nhóm Module AI Kiểm duyệt nội dung

Mã	Tên chức năng	Mô tả
FR-13	Lọc rule-based (Tier 1)	Sau khi bài đăng hoặc bình luận được lưu, tác vụ kiểm duyệt nên áp dụng Regex, danh sách từ khóa và các tín hiệu spam. Nội dung vi phạm rõ ràng được chuyển sang trạng thái từ chối; trường hợp nghi vẫn được phân tích tiếp ở Tier 2
FR-14	Kiểm duyệt AI (Tier 2)	Gọi Gemini API để phân tích nội dung trong các trường hợp: nội dung bị báo cáo, có dấu hiệu vi phạm chưa chắc chắn, hoặc lấy mẫu ngẫu nhiên. API trả về toxic_score, hate_speech_score, spam_score
FR-15	Phân loại kết quả kiểm duyệt	Dựa trên điểm số: $\text{score} < 0.5 \rightarrow$ Approved; $0.5 \leq \text{score} < 0.8 \rightarrow$ Human Review; $\text{score} \geq 0.8 \rightarrow$ Auto Hide
FR-16	Danh sách kiểm duyệt thủ công	Admin xem danh sách nội dung cần Human Review, hiển thị điểm số AI và thông tin liên quan để phê duyệt, ẩn hoặc từ chối nội dung
FR-17	Lưu dấu vết kiểm duyệt	Lưu trạng thái và các điểm toxic, hate-speech, spam trực tiếp trên bài đăng hoặc bình luận; các thao tác quản trị được ghi nhận thông qua cơ chế audit/log của hệ thống

FR-18	Xử lý báo cáo nội dung	Người dùng gửi báo cáo đối với bài đăng, bình luận hoặc tài khoản; quản trị viên xem xét và cập nhật kết quả xử lý
FR-19	Kiểm duyệt media ở mức MVP	Hình ảnh có thể được gửi tới dịch vụ phân tích đa phương thức. Đối với video, hệ thống hiện kiểm tra ảnh thu nhỏ nếu được cung cấp; chưa xử lý chuỗi frame và âm thanh video

Module AI Cá nhân hóa Feed Bảng 3.4: Nhóm Module AI Cá nhân hóa Feed

Mã	Tên chức năng	Mô tả
FR-20	Phân nhánh feed	Hệ thống chia feed thành hai nhánh: Friends Feed (following posts, sắp xếp mới nhất) và Explore Feed (candidate posts từ pool rộng hơn)
FR-21	Lọc điều kiện hiển thị	Lọc bài đăng theo trạng thái hiển thị, danh sách chặn hai chiều, đã xem, và quyền xem bài (canViewPost)
FR-22	Tính điểm ranking (Scoring Engine)	Tính điểm bài đăng dựa trên ba thành phần: Content Matching Score (độ phù hợp chủ đề, hashtag, tác giả với UserProfile), Popularity Score (lượt like, comment, share, save), Social Score (mức độ gần trong social graph). Áp dụng trọng số tĩnh (static weights) và cơ chế cold_start/normal

FR-23	Khám phá nội dung (Exploration)	Chủ động đưa một tỉ lệ bài đăng ngoài vùng sở thích (EXPLORATION_RATE) vào feed để tránh filter bubble
FR-24	Phân trang và enrich	Phân trang kết quả feed; enrich dữ liệu bài đăng với media, hashtag, mention, like/save count
FR-25	Ghi impression	Ghi lại mỗi lần bài đăng hiển thị với người dùng, bao gồm tab, source, position và score breakdown
FR-26	Ghi nhận tương tác người dùng	Ghi nhận các hành vi view, like, comment, share, save, follow-from-post, hide, not-interested và see-more kèm nguồn tương tác, thời gian xem và độ sâu cuộn khi có
FR-27	Cập nhật UserProfile	Dựa trên tương tác, cập nhật topicScores, authorScores, hashtagScores và interaction weight với cơ chế decay theo thời gian
FR-28	Feedback feed	Người dùng có thể phản hồi trực tiếp trên feed: hide, not_interested, see_more, report để điều chỉnh đề xuất

Module giải thích thuật ngữ và ngữ cảnh văn hóa mạng Bảng 3.5: Nhóm Module giải thích thuật ngữ và ngữ cảnh văn hóa mạng

Mã	Tên chức năng	Mô tả
FR-29	Phát hiện thuật ngữ văn hóa mạng	Khi bài đăng được gửi lên, rule-based engine quét nội dung để nhận diện slang, meme, internet culture reference

FR-30	Giải thích hàng loạt qua LLM	Gộp toàn bộ thuật ngữ phát hiện được vào một lần gọi Gemini API duy nhất, lấy giải thích gồm: nghĩa, nguồn gốc, tone cảm xúc, ghi chú ngữ cảnh
FR-31	Lưu kết quả vào bài đăng	Kết quả giải thích được lưu trực tiếp vào document bài đăng trong MongoDB, kèm vị trí ký tự (character offset) của từng thuật ngữ
FR-32	Hiển thị tooltip	Frontend làm nổi bật các thuật ngữ; người đọc nhấn vào để xem phần giải thích đã lưu mà không cần gọi lại Gemini API
FR-33	Phân tích lại khi chỉnh sửa	Khi tác giả chỉnh sửa bài đăng, hệ thống gọi lại pipeline phát hiện và giải thích bất đồng bộ (async) để không chặn luồng lưu bài

Nhắn tin và Thông báo Bảng 3.6: Nhóm Nhắn tin và Thông báo

Mã	Tên chức năng	Mô tả
FR-34	Nhắn tin trực tiếp (DM)	Chat 1-1, 1-N theo thời gian thực qua WebSocket
FR-35	Thông báo hệ thống	Push notification cho các sự kiện: like, comment, follow, tin nhắn

3.2.2. Yêu cầu phi chức năng

Bảng 3.7: Yêu cầu phi chức năng

Mã	Yêu cầu	Mô tả
----	---------	-------

NFR-01	Hiệu năng	Mục tiêu đối với các API không phụ thuộc dịch vụ AI là thời gian phản hồi P95 không vượt quá 500 ms trong điều kiện tải thử nghiệm được xác định. Các tác vụ kiểm duyệt và giải thích thuật ngữ được khởi chạy bất đồng bộ sau khi nội dung được lưu
NFR-02	Khả năng mở rộng	Backend được tổ chức theo kiến trúc monolithic modular. Các module có trách nhiệm tách biệt ở mức mã nguồn, tạo điều kiện cho việc tách thành dịch vụ hoặc bổ sung hàng đợi tác vụ trong các giai đoạn phát triển tiếp theo
NFR-03	Bảo mật	Mật khẩu hash bằng bcrypt. JWT access token thời hạn ngắn kết hợp refresh token. Toàn bộ giao tiếp qua HTTPS/TLS. Validate và sanitize dữ liệu đầu vào để phòng injection
NFR-04	Độ sẵn sàng	Hệ thống đặt mục tiêu uptime 99,5% trong môi trường triển khai. Khi Gemini API timeout, rate limit hoặc trả dữ liệu không hợp lệ, bài đăng vẫn được lưu và hệ thống sử dụng kết quả fallback thay vì trả lỗi cho thao tác đăng bài

NFR-05	Khả năng bảo trì	Logic gọi Gemini và prompt được tập trung trong các service chuyên trách. Trạng thái cùng các điểm kiểm duyệt được lưu trên nội dung; thao tác quản trị được ghi nhận qua cơ chế audit/log để hỗ trợ truy vết
NFR-06	Tính riêng tư	Tuân thủ nguyên tắc tối thiểu hóa dữ liệu. UserProfile (topicScores, authorScores) chỉ dùng nội bộ cho ranking, không chia sẻ ra ngoài. Người dùng có quyền xóa tài khoản và toàn bộ dữ liệu liên quan
NFR-07	Tính tương thích	Mobile app (React Native) hỗ trợ iOS và Android. Web Admin (Next.js) tương thích các trình duyệt hiện đại (Chrome, Firefox, Safari, Edge)

3.2.3. Yêu cầu chất lượng

Các tiêu chí chất lượng được xác định dựa trên các nhóm đặc tính của mô hình chất lượng phần mềm ISO/IEC 25010:

Bảng 3.8: Yêu cầu chất lượng

Đặc tính chất lượng	Tiêu chí đo lường	Mục tiêu
Functional Suitability	Precision, Recall và F1-score của pipeline kiểm duyệt nội dung	Mục tiêu Precision $\geq 85\%$, Recall $\geq 80\%$; kết quả thực tế sẽ được xác định bằng tập dữ liệu gán nhãn

Functional Suitability	Tỉ lệ False Positive của tầng rule-based	Theo dõi và giảm số nội dung an toàn bị phân loại nhầm; giá trị mục tiêu được xác lập sau đợt đánh giá thực nghiệm đầu tiên
Performance Efficiency	Thời gian phản hồi API không phụ thuộc dịch vụ AI	Mục tiêu $P95 \leq 500$ ms trong kịch bản tải thử nghiệm được công bố
Performance Efficiency	Thời gian tải feed, bao gồm scoring và enrich	Theo dõi P50, P95, P99, throughput và tỉ lệ lỗi ở các mức tải 10, 50 và 100 người dùng ảo
Usability	Người dùng mới thực hiện được chức năng cơ bản như đăng bài và xem feed mà không cần hướng dẫn	Mục tiêu ít nhất 90% người tham gia hoàn thành kịch bản trong đợt kiểm thử người dùng
Reliability	Hệ thống vẫn lưu bài khi Gemini API lỗi hoặc phản hồi sai định dạng	100% ca kiểm thử fallback không làm thất bại thao tác tạo bài
Reliability	Độ sẵn sàng của API trong môi trường triển khai	Mục tiêu uptime 99,5%; kết quả được tính theo thời gian giám sát thực tế
Security	Kiểm tra các nhóm rủi ro phổ biến theo OWASP Top 10	Không phát hiện lỗ hổng mức nghiêm trọng trong phạm vi kiểm thử cơ bản

3.3. Kiến trúc hệ thống

3.3.1. Kiến trúc tổng thể

Ở giai đoạn MVP, hệ thống được xây dựng theo mô hình **Monolithic Modular Architecture** — toàn bộ backend nằm trong một project Node.js (Express.js + TypeScript) duy nhất, nhưng được tổ chức thành các module độc lập với trách nhiệm rõ ràng, giao tiếp nội bộ qua lời gọi hàm thay vì mạng. So với kiến trúc Microservices, mô hình này giảm đáng kể độ phức tạp triển khai và chi phí vận hành trong giai đoạn đầu. Việc phân tách trách nhiệm giữa controller, service và model cũng tạo điều kiện thuận lợi hơn nếu cần tách một số module thành dịch vụ độc lập về sau.

Tiêu chí	Monolithic Modular	Microservices
Độ phức tạp triển khai	Thấp — 1 process	Cao — nhiều service, cần orchestration
Chi phí vận hành	Thấp	Cao (container, service mesh)
Độ trễ giao tiếp nội bộ	Không có (function call)	Có (HTTP/gRPC qua mạng)
Khả năng scale từng module	Hạn chế	Linh hoạt
Phù hợp giai đoạn MVP	Cao	Thấp

Các tầng kiến trúc

Hệ thống được phân tách thành bốn tầng như mô tả trong Hình 3.1.

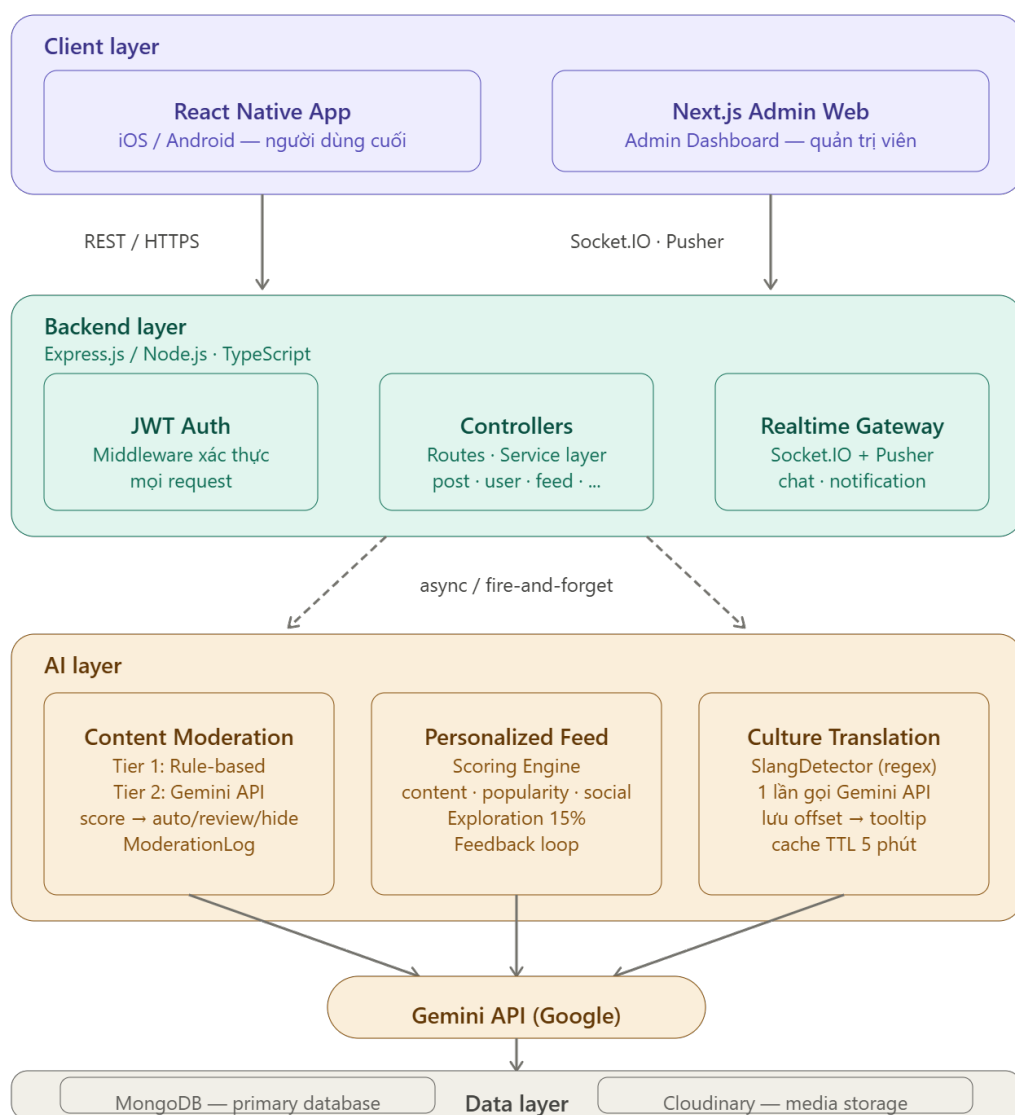
Tầng Client: Gồm hai loại client phục vụ hai nhóm người dùng khác nhau. Ứng dụng di động xây dựng bằng React Native hỗ trợ cả iOS và Android, phục vụ người dùng cuối. Ứng dụng web quản trị (Admin Dashboard) xây dựng bằng Next.js, cung cấp giao diện cho quản trị viên xử lý danh sách báo cáo và nội dung cần xem xét, quản lý từ điển slang và theo dõi hoạt động hệ thống.

Tầng Backend: Server Express.js tiếp nhận toàn bộ request từ client qua REST API. Xác thực được thực hiện qua JWT Middleware trước khi request đến Controller.

Giao tiếp thời gian thực (nhắn tin, thông báo đẩy) được xử lý qua Socket.IO kết hợp Pusher.

Tầng AI Services: Gồm ba module hỗ trợ thông minh chạy bên trong cùng tiến trình Node.js. Sau khi nội dung được lưu, các tác vụ kiểm duyệt và giải thích thuật ngữ được khởi chạy theo cơ chế Promise fire-and-forget. Phiên bản hiện tại chưa sử dụng message queue hoặc worker pool riêng; do đó khả năng phục hồi tác vụ nên khi tiến trình bị gián đoạn vẫn là một hướng cần hoàn thiện.

Tầng Dữ liệu: MongoDB đóng vai trò cơ sở dữ liệu chính lưu toàn bộ dữ liệu hệ thống. Cloudinary được sử dụng để lưu trữ và phân phối hình ảnh, video.



Hình 3.1: Kiến trúc tổng thể của hệ thống

Nguồn: Nhóm tác giả

3.3.2. Module AI Content Moderation

Vấn đề và hướng tiếp cận

Bài toán kiểm duyệt nội dung đặt ra hai yêu cầu cần cân bằng: tốc độ phản hồi và khả năng phân tích ngữ cảnh. Mô hình ngôn ngữ có thể hỗ trợ xử lý các trường hợp phụ thuộc ngữ nghĩa nhưng phát sinh độ trễ và chi phí khi phải gọi API cho mọi nội dung. Trong khi đó, giải pháp thuần rule-based lại có thể bỏ sót các vi phạm phụ thuộc ngữ cảnh. Hệ thống cân bằng hai hướng tiếp cận bằng kiến trúc **2-Tier Pipeline**, kết hợp rule-based filter và phân tích bằng mô hình ngôn ngữ.

Phương án	Tốc độ	Chi phí	Khả năng phân tích ngữ cảnh	Phù hợp
Thuần Rule-based	Nhanh	Không phát sinh chi phí API	Hạn chế	Không
Thuần LLM API	Phụ thuộc dịch vụ ngoài	Phát sinh theo số lần gọi	Có	Không
2-Tier Hybrid	Có fast path	Giảm số lần gọi API	Kết hợp hai nguồn tín hiệu	Phù hợp

Tier 1 — Rule-based Filter

RuleBasedService thực hiện các phép kiểm tra đồng bộ bên trong tác vụ kiểm duyệt. Tuy nhiên, khi tạo bài đăng, toàn bộ tác vụ kiểm duyệt được khởi chạy sau khi bài viết đã được lưu và không được await trong request chính. Module thực hiện ba lớp kiểm tra theo thứ tự ưu tiên, dừng lại và trả kết quả ngay khi phát hiện vi phạm rõ ràng ở bất kỳ lớp nào.

Lớp 1 — Profanity Blocklist: Danh sách 100+ từ/cụm từ độc hại (tiếng Việt và tiếng Anh) được pre-compiled tại thời điểm khởi động thành BlocklistEntry với regex

kèm word-boundary. Kỹ thuật word-boundary `(?:^\s)word(?:\s|$)` giải quyết vấn đề false positive phổ biến khi từ vi phạm xuất hiện bên trong một từ khác dài hơn — ví dụ, từ ”đụ” không được match nằm trong ”đụng” hay ”đụt”. Văn bản đầu vào được chuẩn hóa trước (lowercase, bỏ dấu NFD, thay dấu câu bằng khoảng trắng) để đảm bảo match nhất quán bất kể cách viết của người dùng. Nếu khớp, hàm trả về `isClearViolation = true`, `score = 0.95`.

Lớp 2 — Hate-speech Pattern: Bốn nhóm regex mô tả cấu trúc câu thù hận đặc trưng trong tiếng Việt (đe dọa, miệt thị, kêu gọi bạo lực). Mỗi regex được khởi tạo lại thông qua `new RegExp(pattern.source, pattern.flags)` trước mỗi lần kiểm tra, thay vì tái sử dụng cùng một object. Đây là biện pháp phòng tránh lỗi `lastIndex` — một đặc tính của JavaScript khi regex mang flag `/g` lưu trạng thái vị trí tìm kiếm giữa các lần gọi `.test()`. Nếu tái sử dụng cùng một regex object, lần gọi thứ ba hoặc thứ tư có thể trả về false trên cùng một chuỗi vi phạm, dẫn đến bỏ sót nội dung độc hại một cách ngẫu nhiên. Nếu khớp, hàm trả về `isClearViolation = true`, `score = 0.92`.

Lớp 3 — Spam Signal: Nếu nội dung vượt qua Lớp 1 và Lớp 2, hệ thống tính điểm spam tổng hợp từ ba tín hiệu độc lập: ký tự lặp liên tiếp từ 8 lần, nhiều link trong cùng một bài (≥ 2 URL), và toàn chữ hoa trên đoạn văn dài ($> 85\%$ uppercase trên > 40 ký tự). Khi có nhiều tín hiệu cùng xuất hiện, điểm được cộng dồn có giới hạn theo công thức:

$$spamScore = \min(maxSignalScore + 0.2 \times (signalCount - 1), 0.9)$$

Kết quả Lớp 3 phân thành ba nhánh: `spamScore  $\geq 0.70$` thì `isClearViolation = true` — chặn ngay; $0.25 \leq spamScore < 0.70$ thì `needsAICheck = true` — vùng nghi ngờ, chuyển sang Tier 2; `spamScore < 0.25` — nội dung sạch, duyệt ngay theo fast path.

Tier 2 — AI Moderation

`ModerationService` nhận kết quả từ `RuleBasedService` và xử lý theo ba nhánh chính. Nếu `isClearViolation = true` (từ bất kỳ lớp nào của Tier 1), nội dung bị chặn ngay lập tức mà không gọi Gemini API. Nếu hàm `shouldCallAI()` trả về true, hệ thống gọi Tier 2. Trường hợp còn lại, nội dung được duyệt ngay theo fast path.

Tier 2 được kích hoạt khi thỏa một trong bốn điều kiện: `needsAICheck = true`

từ Lớp 3, nội dung bị người dùng báo cáo (`reportCount > 0`), tài khoản mới (`isNewAccount = true`), hoặc lấy mẫu ngẫu nhiên 5% để kiểm soát chất lượng hệ thống. `AIAPIService` gọi Gemini API với prompt được thiết kế ép buộc mô hình trả về JSON chuẩn:

```
{
  "toxic": number 0-1,
  "hate_speech": number 0-1,
  "spam": number 0-1,
  "reason": string
}
```

Sau khi nhận kết quả từ Gemini, hệ thống không dùng điểm AI thuần túy mà tổng hợp với điểm từ Tier 1 bằng phép lấy MAX trên từng chiều đánh giá:

```
combinedToxic = max(aiScores.toxic, ruleScore × 0.35)
combinedHateSpeech = max(aiScores.hateSpeech, ruleScore × 0.3)
combinedSpam = max(aiScores.spam, ruleScore)
```

Cơ chế này đảm bảo tín hiệu nghi ngờ từ Tier 1 không bị kết quả AI lạc quan che khuất — nếu rule-based đã thấy dấu hiệu spam (`score = 0.4`) nhưng Gemini đánh giá sạch (`spam = 0.1`), điểm spam cuối cùng vẫn giữ nguyên là 0.4. Quyết định cuối cùng dựa trên `maxScore = max(toxic, hateSpeech, spam)` theo ngưỡng phân loại trong Bảng 3.3:

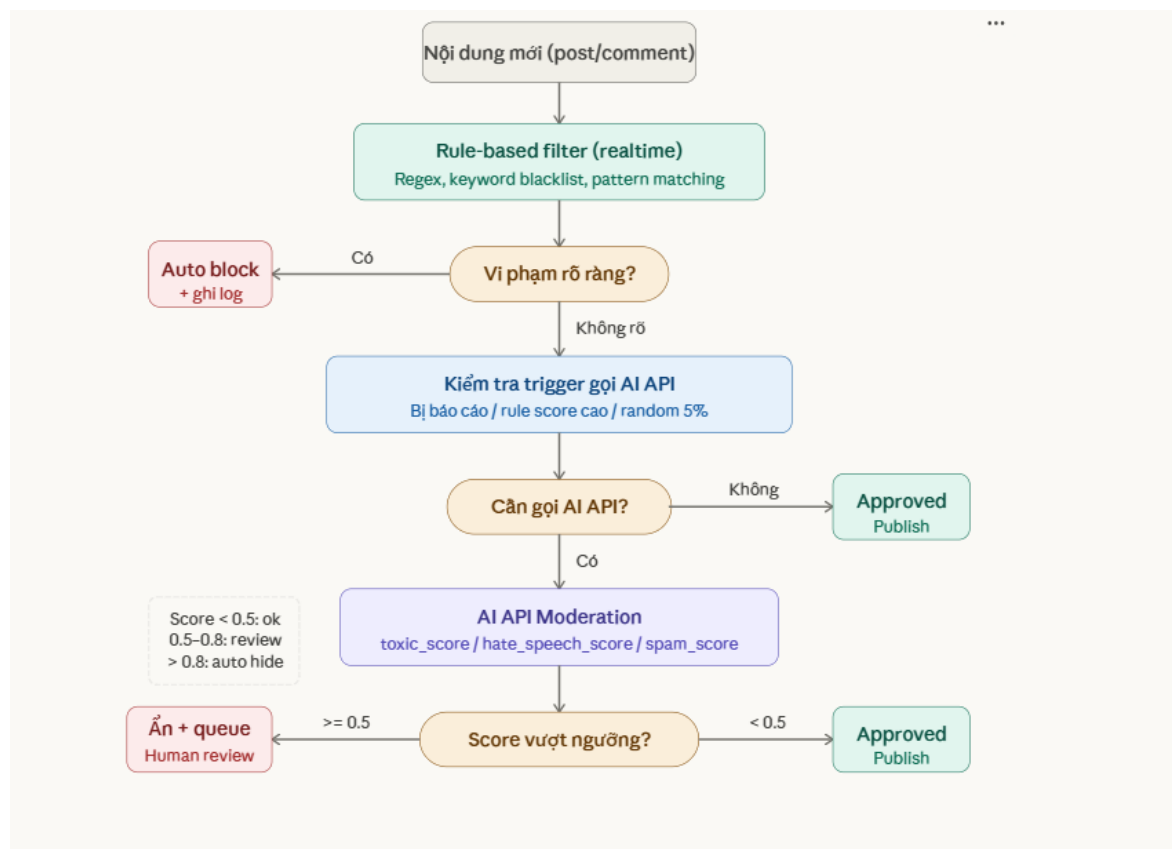
maxScore	Quyết định	Trạng thái	Hành động
< 0.5	Approved	APPROVED	Cho phép hiển thị
0.5 – < 0.8	Human Review	FLAGGED	Đưa vào danh sách cần xem xét
>= 0.8	Auto Hide	REJECTED	Tự động ẩn, ghi hiddenReason

Cơ chế Fallback

Hệ thống xử lý đầy đủ các trường hợp lỗi API (timeout, rate limit, JSON parse error) theo nguyên tắc: bài đăng không bao giờ bị mất do lỗi AI. Khi Gemini API thất bại do lỗi HTTP hoặc exception, `AIAPIService` trả về điểm mặc định thấp (`toxic: 0.2, hateSpeech: 0, spam: 0.2`) kèm lý do `fallback_*`. Với `maxScore = 0.2 < 0.5`, nội dung được phân loại APPROVED theo ngưỡng hiện tại; thao tác tạo bài không bị trả lỗi. Trường hợp parse JSON thất bại sử dụng bộ điểm fallback thấp hơn. Trạng thái và các điểm kiểm duyệt được lưu trực tiếp trên document bài đăng hoặc bình luận; các

thao tác quản trị được ghi nhận qua cơ chế audit/log của hệ thống.

Hình 3.2 mô tả luồng xử lý đầy đủ của 2-Tier Moderation Pipeline:



Hình 3.2: Luồng xử lý của 2-Tier Moderation Pipeline

Nguồn: Nhóm tác giả

3.3.3. Module Personalized Feed

Vấn đề và hướng tiếp cận

Hệ thống đề xuất nội dung thuần content-based filtering có xu hướng dẫn đến ”overspecialization” — người dùng bị mắc kẹt trong vòng lặp nội dung hẹp [9]. Hệ thống giải quyết vấn đề này bằng kiến trúc **Hybrid Scoring** kết hợp ba chiều đánh giá độc lập (content, popularity, social), bổ sung cơ chế Exploration chủ động đưa nội dung ngoài vùng sở thích vào feed.

Scoring Engine

ScoringService tính điểm ranking cho mỗi bài đăng theo công thức tổng hợp có trọng số:

$$finalScore = w_1 \times contentScore + w_2 \times popularityScore + w_3 \times socialScore$$

Trọng số được điều chỉnh tự động dựa trên lượng dữ liệu hành vi của người dùng, phân biệt hai chế độ theo Bảng 3.9:

Bảng 3.9: Trọng số Scoring Engine theo chế độ người dùng

Thành phần	Normal (≥ 10 tương tác)	Cold Start (< 10 tương tác)	Lý do
Content (w_1)	0.50	0.05	Người dùng mới chưa có đủ profile
Popularity (w_2)	0.20	0.7	Nội dung phổ biến phù hợp khi thiếu dữ liệu
Social (w_3)	0.30	0.25	Ưu tiên người đang follow

Content Score đo độ khớp giữa bài đăng và UserProfile của người dùng. TopicExtractorService trích xuất tối đa 3 chủ đề từ bài đăng bằng cách tính điểm keyword matching trên hai tầng: hashtag (trọng số $\times 2$) và từ khóa trong nội dung (trọng số $\times 1$), đối chiếu với từ điển social_media_topics_v2.json gồm các chủ đề tiếng Việt lẫn tiếng Anh. Content Score được tính:

$$contentScore = \sum topicScores[topic] + authorScores[authorId] \times 1.2$$

Popularity Score sử dụng trường hotScore đã được tính sẵn bởi cron job định kỳ 30 phút. Khi hotScore chưa có, hệ thống tính tạm theo công thức có tham số gravity kiểm soát tốc độ suy giảm theo thời gian:

$$rawPopularity = engagements / (ageHours + 2)^{gravity}$$

$$popularityScore = \min(rawPopularity \times 2, 100)$$

Hệ số nhân 2 được khai báo tập trung và cùng hàm calculateHotScore được sử dụng cho cả cron job lẫn nhánh tính dự phòng, bảo đảm hai nhánh có cùng thang điểm. Trong đó $gravity = 1.5$ và:

$$engagements = likes \times 3 + comments \times 4 + shares \times 5 + saves \times 4 + views \times 0.1$$

Social Score nhị phân: bài đăng từ tác giả đang được follow nhận điểm tối đa (100), ngược lại nhận 0. Tính đơn giản nhưng hiệu quả trong việc ưu tiên nội dung từ

mạng lưới gần.

Exploration

Để giảm nguy cơ filter bubble, hệ thống dành 15% vị trí trong feed cho các bài viết không giao với nhóm chủ đề có điểm sở thích hiệu lực cao nhất của người dùng. Các ứng viên ngoài vùng sở thích được ưu tiên lấy từ phân xếp hạng thấp hơn và được phân bố xen kẽ trong feed thay vì dồn vào cuối danh sách. Khi không có đủ ứng viên ngoài vùng sở thích, hệ thống sử dụng các ứng viên ở phần cuối bảng xếp hạng làm phương án dự phòng và không lặp bài.

Feedback Loop

InteractionTrackerService thu thập tín hiệu hành vi người dùng và cập nhật liên tục vào UserProfile. Mỗi loại tương tác mang trọng số và hệ số cường độ tín hiệu khác nhau theo Bảng 3.10:

Bảng 3.10: Trọng số và hệ số tín hiệu theo loại tương tác

Loại tương tác	Trọng số (weight)	Hệ số tín hiệu	Ý nghĩa
follow_from_post	6	1.2	Tín hiệu mạnh nhất — chủ động theo dõi
share	5	1.0	Muốn lan truyền nội dung
comment	4	1.0	Tương tác sâu
save	4	1.0	Muốn xem lại
like	3	1.0	Thích nội dung
see_more	2	0.8	Muốn xem thêm về chủ đề
view	1	0.6	Tín hiệu yếu — có thể vô tình
not_interested	-3	1.0	Phản hồi tiêu cực nhẹ
hide	-5	1.0	Phản hồi tiêu cực mạnh

Delta của một tương tác được tính:

$$\text{delta} = \text{weight} \times \text{signalMultiplier}$$

Mỗi giá trị trong `topicScores`, `hashtagScores` và `authorScores` được lưu riêng dưới dạng cặp $\{\text{score}, \text{lastUpdatedAt}\}$. Trước khi cộng tín hiệu mới vào một key liên quan đến bài viết, hệ thống tính điểm đã suy giảm theo thời gian:

$$\text{decayedScore} = \text{oldScore} \times 0.98^{\text{daysSinceLastUpdate}}$$

Sau đó hệ thống cộng $\text{delta} \times 1.0$ cho topic, $\text{delta} \times 0.9$ cho hashtag và $\text{delta} \times 0.7$ cho tác giả, đồng thời cập nhật `lastUpdatedAt` của đúng key đó. Các key không liên quan đến tương tác hiện tại không bị thay đổi timestamp. Khi xếp hạng feed, `ScoringService` cũng tính điểm hiệu lực từ timestamp riêng của từng key, nhờ đó sở thích cũ giảm dần ngay cả khi người dùng gần đây tương tác với một chủ đề khác.

Các bản ghi hành vi thô được lưu trong `UserInteraction` với TTL index tự động xóa sau 90 ngày. TTL kiểm soát thời gian lưu lịch sử tương tác, trong khi cơ chế suy giảm theo từng key kiểm soát mức độ ảnh hưởng lâu dài của các điểm đã tổng hợp trong `UserProfile`.

Hình 3.3 mô tả pipeline đầy đủ của Personalized Feed:



Hình 3.3: Luồng xử lý của Personalized Feed

Nguồn: Nhóm tác giả

3.3.4. Module giải thích thuật ngữ và ngữ cảnh văn hóa mạng

Vấn đề và hướng tiếp cận

Ngôn ngữ mạng (slang, meme, internet culture reference) phát triển nhanh chóng tạo ra rào cản giao tiếp giữa các thế hệ và nhóm văn hóa. Các nền tảng hiện tại chưa có cơ chế giải thích ngữ cảnh văn hóa, dẫn đến hiểu nhầm và giảm khả năng tiếp cận của người dùng lớn tuổi [6]. Phương án gọi LLM mỗi khi người dùng hover vào từ sẽ gây độ trễ cao và chi phí không kiểm soát được. Hệ thống giải quyết bằng chiến lược **phân tích một lần tại thời điểm đăng bài** — kết quả được lưu vào MongoDB và phục vụ trực tiếp từ database khi người đọc tương tác.

Pipeline xử lý

Toàn bộ pipeline chạy bất đồng bộ (fire-and-forget) sau khi bài đăng đã được lưu thành công, không chặn luồng phản hồi của người dùng.

Bước 1 — Rule-based Detection (SlangDetectorService): SlangDetectorService tải từ điển slang từ SlangEntryModel trong MongoDB với cơ chế in-memory cache TTL 5 phút, tránh truy vấn DB lặp lại cho mỗi bài đăng. Với mỗi entry trong từ điển, regex được build từ trường regexPattern lưu trong DB và thực thi với flag /gi để tìm tất cả vị trí xuất hiện trong nội dung. Kết quả trả về danh sách DetectedTerm kèm vị trí ký tự chính xác (startIndex, endIndex) — dữ liệu cần thiết để frontend highlight đúng vị trí trong chuỗi gốc. Bước này hoàn toàn không tốn chi phí API.

Bước 2 — Một lần gọi LLM (CultureLLMService): Thay vì gọi API riêng lẻ cho từng thuật ngữ, toàn bộ danh sách thuật ngữ phát hiện được gộp vào **một lần gọi Gemini API duy nhất**. Đây là điểm tối ưu chi phí quan trọng nhất của module — giả sử một bài đăng chứa 5 thuật ngữ slang, phương án gọi riêng lẻ tốn 5 lần API, phương án batch chỉ tốn 1 lần.

Prompt được thiết kế cung cấp ngữ cảnh bài viết (300 ký tự đầu) để LLM giải thích đúng nghĩa trong văn cảnh cụ thể, không chỉ nghĩa từ điển chung. Đầu ra yêu cầu JSON array chuẩn, mỗi phần tử gồm: term, meaning (giải thích tối đa 2 câu), origin (TikTok / Reddit VN / Gaming / Twitter / Chung), tone (tích cực / trung tính / hài hước / tiêu cực), contextNote (cách dùng trong bài này). Tương tự module Moderation, mọi trường hợp lỗi API đều có fallback trả về cấu trúc rỗng, không làm crash pipeline.

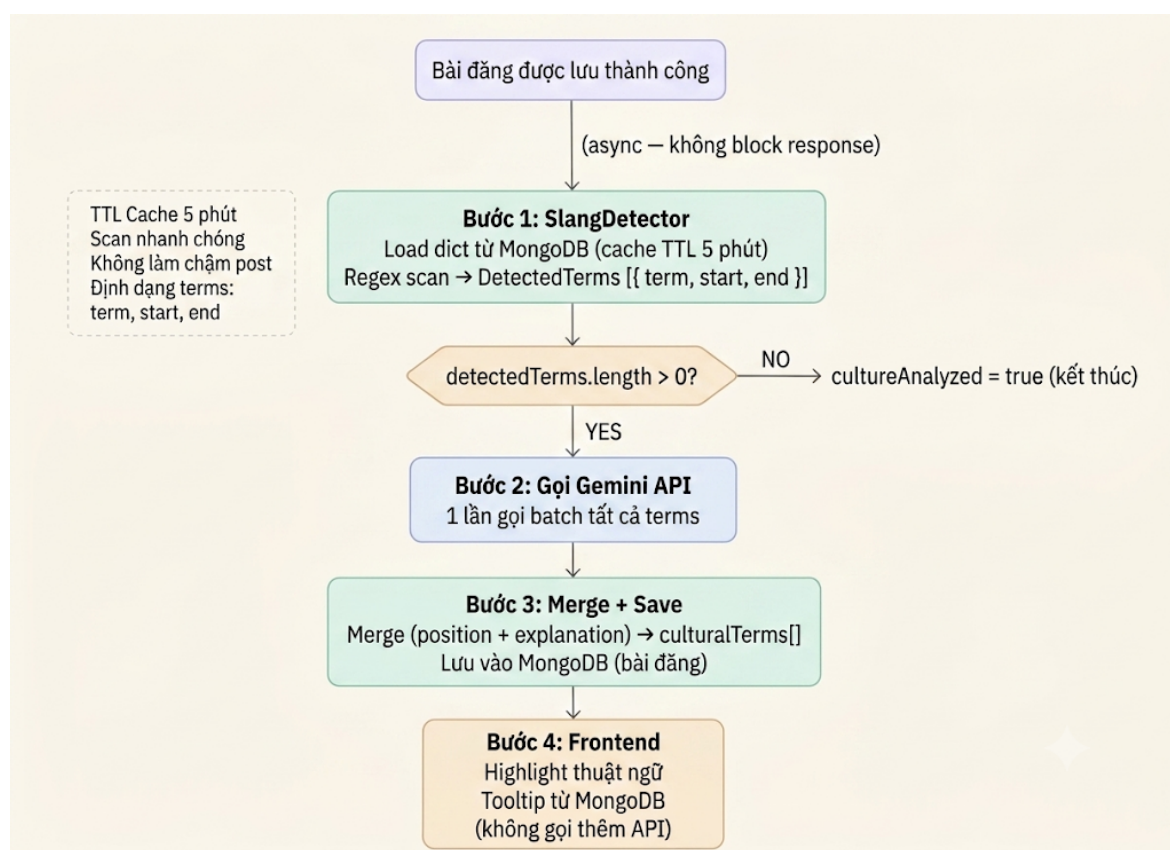
Bước 3 — Merge và lưu MongoDB: Kết quả giải thích từ LLM được ghép với vị trí ký tự từ Bước 1 thông qua explainMap (Map theo term lowercase). Dữ liệu culturalTerms được lưu trực tiếp vào document bài đăng kèm cờ cultureAnalyzed = true để tránh xử lý lại.

Bước 4 — Hiển thị Tooltip (Frontend): Khi người đọc mở bài đăng, client nhận culturalTerms kèm startIndex/endIndex, tự động highlight các thuật ngữ trong văn bản. Người dùng nhấn vào thuật ngữ được highlight sẽ thấy tooltip giải thích từ dữ liệu đã lưu — không phát sinh thêm lần gọi Gemini API. Cách tiếp cận này loại bỏ độ trễ gọi LLM tại thời điểm người đọc mở phần giải thích, dù thời gian hiển thị thực tế vẫn phụ thuộc vào mạng và quá trình tải dữ liệu từ backend.

Xử lý khi chỉnh sửa bài

Khi tác giả chỉnh sửa nội dung, reAnalyzePost reset culturalTerms về mảng rỗng và cultureAnalyzed về false, sau đó gọi lại toàn bộ pipeline bất đồng bộ. Cơ chế này đảm bảo kết quả phân tích luôn khớp với nội dung hiện tại của bài đăng.

Hình 3.4 mô tả luồng xử lý của Culture Translation Pipeline:



Hình 3.4: Luồng giải thích thuật ngữ và ngữ cảnh văn hóa mạng

Nguồn: Nhóm tác giả

3.4. Thiết kế Use Case

3.4.1. Danh sách các Actor

Bảng 3.9: Danh sách các actor của hệ thống

Mã actor	Tên actor	Mô tả vai trò
ACT-01	Khách	Người chưa đăng nhập, có thể đăng ký tài khoản, đăng nhập, xác thực OTP trước đăng ký và khôi phục mật khẩu.
ACT-02	Người dùng	Người dùng đã xác thực, sử dụng đầy đủ các chức năng mạng xã hội như quản lý hồ sơ, bài viết, tương tác, nhắn tin, nhận thông báo và báo cáo nội dung.
ACT-03	Quản trị viên	Người dùng có quyền quản trị, xử lý báo cáo, quản lý người dùng, bài viết, bình luận, từ điển slang và theo dõi chỉ số vận hành hệ thống.
ACT-04	Gemini API	Dịch vụ mô hình ngôn ngữ lớn được hệ thống gọi để hỗ trợ kiểm duyệt nội dung, giải thích thuật ngữ văn hóa mạng và suy luận ngữ cảnh.
ACT-05	Dịch vụ OTP/Email/SMS	Nhóm dịch vụ ngoài gồm SMTP, Twilio hoặc cơ chế gửi OTP tương đương, chịu trách nhiệm phát hành và gửi mã xác thực.
ACT-06	Google OAuth	Dịch vụ xác thực bên thứ ba hỗ trợ đăng nhập nhanh bằng tài khoản Google.
ACT-07	Cloudinary	Dịch vụ lưu trữ và phân phối ảnh, video, tệp đính kèm cho bài viết, hồ sơ và tin nhắn.
ACT-08	Pusher/Socket.IO	Hạ tầng giao tiếp thời gian thực phục vụ nhắn tin, trạng thái trực tuyến và phân phối thông báo gần thời gian thực.

3.4.2. Danh sách các Use Case chính

Hệ thống được phân rã thành các nhóm use case lớn tương ứng với các mô-đun nghiệp vụ đã triển khai trong backend. Script PlantUML dùng để vẽ sơ đồ use case tổng thể được lưu tại tệp `report/usecase-system.puml`.

Bảng 3.10: Danh sách các use case chính

Mã use case	Tên use case	Actor chính	Mục tiêu nghiệp vụ
UC-01	Đăng ký tài khoản	Khách	Tạo tài khoản mới bằng email hoặc số điện thoại sau khi hoàn tất bước xác thực OTP trước đăng ký.
UC-02	Đăng nhập và duy trì phiên	Khách, Người dùng	Cho phép người dùng đăng nhập, làm mới phiên, đăng xuất và tiếp tục đăng nhập với 2FA khi được bật.
UC-03	Quản lý xác thực đa yếu tố	Người dùng	Thiết lập, kích hoạt hoặc tắt xác thực hai yếu tố để tăng cường bảo mật tài khoản.
UC-04	Khôi phục mật khẩu	Khách	Yêu cầu đặt lại mật khẩu khi quên thông tin đăng nhập.
UC-05	Quản lý hồ sơ cá nhân	Người dùng	Xem và cập nhật hồ sơ, ảnh đại diện, ảnh nền và thông tin cá nhân.
UC-06	Tìm kiếm toàn cục	Người dùng	Tìm kiếm người dùng, bài viết và hashtag phục vụ khám phá nội dung.

Mã use case	Tên use case	Actor chính	Mục tiêu nghiệp vụ
UC-07	Quản lý quan hệ người dùng	Người dùng	Gửi yêu cầu theo dõi, chấp nhận hoặc từ chối yêu cầu, quản lý close friend, chặn và bỏ chặn.
UC-08	Quản lý bài viết	Người dùng	Tạo, xem, sửa, xóa và truy xuất bài viết theo người dùng hoặc theo ngữ cảnh hiển thị.
UC-09	Tương tác với nội dung	Người dùng	Thích, bỏ thích, lưu, bỏ lưu, bình luận, trả lời bình luận, chia sẻ, repost và báo cáo nội dung hoặc tài khoản.
UC-10	Xem feed cá nhân hóa	Người dùng	Nhận feed Friends hoặc Explore được xếp hạng dựa trên hồ sơ hành vi và phản hồi của người dùng.
UC-11	Giải thích thuật ngữ văn hóa mạng	Người dùng	Xem các thuật ngữ được phát hiện trong bài viết, nhận giải thích theo ngữ cảnh và báo lỗi thuật ngữ nếu cần.
UC-12	Nhận và quản lý thông báo	Người dùng	Đăng ký thiết bị, xem thông báo, đánh dấu đã đọc hoặc đã xem và xóa thông báo.

Mã use case	Tên use case	Actor chính	Mục tiêu nghiệp vụ
UC-13	Nhắn tin và gọi thoại	Người dùng	Trao đổi tin nhắn trực tiếp hoặc nhóm, quản lý nhóm chat, file đính kèm, trạng thái đọc và lịch sử cuộc gọi.
UC-14	Gửi báo cáo vi phạm	Người dùng	Gửi báo cáo đối với bài viết, bình luận, tài khoản hoặc thuật ngữ sai để hệ thống và quản trị viên xử lý.
UC-15	Kiểm duyệt nội dung bằng AI	Hệ thống, Gemini API, Quản trị viên	Phân tích nội dung theo pipeline 2 tầng, tự động duyệt, gắn cờ hoặc ẩn nội dung vi phạm.
UC-16	Quản trị báo cáo và nội dung	Quản trị viên	Xem hàng đợi báo cáo, xử lý bài viết, bình luận, tài khoản bị báo cáo và ghi nhận kết quả xử lý.
UC-17	Quản lý từ điển slang	Quản trị viên	Thêm mới hoặc bật tắt các mục trong từ điển slang phục vụ phát hiện thuật ngữ văn hóa mạng.
UC-18	Theo dõi thống kê hệ thống	Quản trị viên	Xem dashboard, chỉ số AI, thống kê người dùng, audit log và dữ liệu vận hành.

3.4.3. Đặc tả chi tiết Use Case

Phần này đặc tả chi tiết các use case chính ở mức nghiệp vụ của toàn bộ hệ thống. Mỗi use case được mô tả theo cùng một khung gồm mục tiêu, actor, tiền điều kiện, hậu điều kiện, luồng chính và các luồng thay thế hoặc ngoại lệ để thuận tiện cho việc thiết kế API, giao diện và kiểm thử.

UC-01: Đăng ký tài khoản **Mục tiêu:** Cho phép khách tạo tài khoản mới bằng email hoặc số điện thoại hợp lệ.

Actor chính: Khách.

Tiền điều kiện: Người dùng chưa có tài khoản với định danh đang dùng; OTP trước đăng ký còn hiệu lực.

Hậu điều kiện: Tài khoản mới được tạo và sẵn sàng cho đăng nhập hoặc xác thực bổ sung.

Luồng chính:

1. Khách nhập thông tin đăng ký gồm tên hiển thị, email hoặc số điện thoại, mật khẩu và các trường bắt buộc.
2. Hệ thống yêu cầu gửi OTP trước đăng ký đến email hoặc số điện thoại đã khai báo.
3. Dịch vụ OTP/Email/SMS gửi mã xác thực cho khách.
4. Khách nhập mã OTP đã nhận.
5. Hệ thống kiểm tra mã OTP, thời hạn và tần suất yêu cầu.
6. Nếu hợp lệ, hệ thống tạo tài khoản người dùng mới và lưu thông tin đăng ký.
7. Hệ thống trả kết quả đăng ký thành công cho client.

Luồng thay thế / ngoại lệ:

1. Email hoặc số điện thoại đã tồn tại trong hệ thống, hệ thống từ chối tạo tài khoản.
2. OTP sai, hết hạn hoặc vượt giới hạn thử lại, hệ thống yêu cầu gửi mã mới.

3. Dịch vụ gửi OTP lỗi, hệ thống trả thông báo thất bại và không tạo tài khoản.

UC-02: Đăng nhập và duy trì phiên **Mục tiêu:** Cho phép người dùng đăng nhập, làm mới access token, đăng xuất và hoàn tất đăng nhập qua 2FA nếu được bật.

Actor chính: Khách, Người dùng.

Tiền điều kiện: Tài khoản tồn tại và chưa bị khóa hoặc vô hiệu hóa.

Hậu điều kiện: Người dùng có phiên đăng nhập hợp lệ hoặc phiên hiện tại bị hủy khi đăng xuất.

Luồng chính:

1. Khách nhập email hoặc số điện thoại cùng mật khẩu, hoặc chọn đăng nhập Google.
2. Hệ thống xác thực thông tin đăng nhập.
3. Nếu tài khoản không bật 2FA, hệ thống phát hành access token và refresh token.
4. Nếu tài khoản bật 2FA, hệ thống yêu cầu mã xác thực bổ sung.
5. Người dùng nhập mã 2FA.
6. Hệ thống kiểm tra mã và phát hành bộ token nếu hợp lệ.
7. Trong quá trình sử dụng, client có thể gọi chức năng refresh token để duy trì phiên.
8. Khi người dùng đăng xuất, hệ thống thu hồi hoặc xóa refresh token hiện hành.

Luồng thay thế / ngoại lệ:

1. Sai mật khẩu hoặc tài khoản không tồn tại, hệ thống từ chối đăng nhập.
2. Mã 2FA sai hoặc hết hạn, hệ thống không cấp phiên mới.
3. Refresh token không tồn tại hoặc không hợp lệ, hệ thống yêu cầu đăng nhập lại.

UC-03: Quản lý xác thực đa yếu tố **Mục tiêu:** Cho phép người dùng thiết lập, bật hoặc tắt cơ chế xác thực hai yếu tố.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập hợp lệ.

Hậu điều kiện: Trạng thái 2FA của tài khoản được cập nhật đúng theo thao tác người dùng.

Luồng chính:

1. Người dùng truy cập chức năng bảo mật tài khoản.
2. Hệ thống tạo thông tin thiết lập 2FA, ví dụ secret hoặc mã QR tương ứng.
3. Người dùng quét hoặc nhập secret vào ứng dụng xác thực.
4. Người dùng nhập mã OTP sinh ra từ ứng dụng xác thực để kích hoạt.
5. Hệ thống kiểm tra mã và cập nhật trạng thái 2FA là đang bật.
6. Khi cần tắt 2FA, người dùng gửi yêu cầu vô hiệu hóa.
7. Hệ thống kiểm tra lại điều kiện an toàn và cập nhật trạng thái 2FA.

Luồng thay thế / ngoại lệ:

1. Người dùng nhập mã kích hoạt sai, hệ thống không bật 2FA.
2. Người dùng chưa đăng nhập hoặc phiên hết hạn, hệ thống từ chối thao tác.
3. Thao tác tắt 2FA không thỏa điều kiện xác minh hiện hành, hệ thống yêu cầu xác thực lại.

UC-04: Khôi phục mật khẩu **Mục tiêu:** Hỗ trợ người dùng đặt lại mật khẩu khi quên thông tin đăng nhập.

Actor chính: Khách.

Tiền điều kiện: Tài khoản liên kết với email hoặc số điện thoại có thể nhận mã xác minh.

Hậu điều kiện: Mật khẩu mới được cập nhật và mật khẩu cũ không còn hiệu lực.

Luồng chính:

1. Khách chọn chức năng quên mật khẩu.
2. Hệ thống yêu cầu khách nhập email hoặc số điện thoại.
3. Hệ thống tạo mã xác minh hoặc token đặt lại mật khẩu.
4. Dịch vụ OTP/Email/SMS gửi thông tin khôi phục cho khách.
5. Khách nhập mã xác minh và mật khẩu mới.
6. Hệ thống kiểm tra mã xác minh và chính sách mật khẩu.
7. Hệ thống cập nhật mật khẩu mới cho tài khoản.

Luồng thay thế / ngoại lệ:

1. Mã xác minh sai hoặc hết hạn, hệ thống từ chối đổi mật khẩu.
2. Định danh không tồn tại, hệ thống trả thông báo tổng quát để tránh lộ thông tin tài khoản.
3. Mật khẩu mới không đạt yêu cầu độ mạnh, hệ thống yêu cầu nhập lại.

UC-05: Quản lý hồ sơ cá nhân **Mục tiêu:** Cho phép người dùng xem và cập nhật hồ sơ cá nhân.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập.

Hậu điều kiện: Thông tin hồ sơ, ảnh đại diện hoặc ảnh nền được cập nhật trong hệ thống.

Luồng chính:

1. Người dùng mở trang hồ sơ cá nhân.
2. Hệ thống trả về thông tin hồ sơ hiện tại.
3. Người dùng chỉnh sửa thông tin mô tả, tên hiển thị hoặc các trường cho phép.
4. Người dùng có thể tải ảnh đại diện hoặc ảnh nền mới.

5. Hệ thống tải tệp lên Cloudinary và nhận URL lưu trữ.
6. Hệ thống cập nhật hồ sơ người dùng và trả dữ liệu mới cho client.

Luồng thay thế / ngoại lệ:

1. Tệp tải lên không hợp lệ hoặc vượt giới hạn, hệ thống từ chối cập nhật ảnh.
2. Phiên đăng nhập hết hạn, hệ thống yêu cầu người dùng đăng nhập lại.

UC-06: Tìm kiếm toàn cục **Mục tiêu:** Cho phép người dùng tìm kiếm người dùng, bài viết và hashtag trong hệ thống.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập.

Hậu điều kiện: Danh sách kết quả phù hợp được hiển thị cho người dùng.

Luồng chính:

1. Người dùng nhập từ khóa tìm kiếm.
2. Hệ thống nhận truy vấn và chuẩn hóa chuỗi tìm kiếm.
3. Hệ thống tìm kiếm đồng thời trên các thực thể người dùng, bài viết và hashtag.
4. Hệ thống tổng hợp, sắp xếp và trả về kết quả cho client.
5. Người dùng chọn một kết quả để xem chi tiết hoặc điều hướng đến màn hình liên quan.

Luồng thay thế / ngoại lệ:

1. Từ khóa rỗng hoặc không hợp lệ, hệ thống yêu cầu nhập lại.
2. Không có kết quả phù hợp, hệ thống trả danh sách rỗng.

UC-07: Quản lý quan hệ người dùng **Mục tiêu:** Hỗ trợ người dùng xây dựng và kiểm soát quan hệ theo dõi, close friend và danh sách chặn.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập và tài khoản đích tồn tại.

Hậu điều kiện: Trạng thái quan hệ giữa hai người dùng được cập nhật chính xác.

Luồng chính:

1. Người dùng gửi yêu cầu theo dõi tới một tài khoản khác.
2. Nếu tài khoản đích ở chế độ riêng tư, hệ thống tạo yêu cầu chờ phản hồi.
3. Tài khoản đích chấp nhận hoặc từ chối yêu cầu theo dõi.
4. Khi quan hệ theo dõi hình thành, người dùng có thể bỏ theo dõi hoặc xóa follower.
5. Người dùng có thể gửi yêu cầu close friend cho một quan hệ phù hợp.
6. Người dùng có thể chặn hoặc bỏ chặn tài khoản khác khi cần.
7. Hệ thống cập nhật thống kê follower, following và danh sách liên quan.

Luồng thay thế / ngoại lệ:

1. Người dùng gửi yêu cầu trùng lặp, hệ thống từ chối thao tác.
2. Tài khoản đang nằm trong danh sách chặn, hệ thống không cho tạo quan hệ mới.
3. Người dùng không đủ quyền chấp nhận hoặc từ chối yêu cầu không thuộc về mình.

UC-08: Quản lý bài viết **Mục tiêu:** Cho phép người dùng tạo, xem, sửa và xóa bài viết cùng nội dung đa phương tiện.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập và tài khoản còn hoạt động.

Hậu điều kiện: Bài viết được tạo mới, cập nhật hoặc xóa theo thao tác.

Luồng chính:

1. Người dùng nhập nội dung bài viết, hashtag, mention, quyền riêng tư và media đính kèm nếu có.
2. Hệ thống tải media lên Cloudinary nếu bài viết có ảnh hoặc video.

3. Hệ thống lưu bài viết vào cơ sở dữ liệu.
4. Sau khi lưu thành công, hệ thống khởi chạy các pipeline nền như kiểm duyệt nội dung và phân tích thuật ngữ.
5. Người dùng có thể xem danh sách bài viết, chi tiết bài viết hoặc bài viết theo hồ sơ người dùng.
6. Tác giả có thể chỉnh sửa hoặc xóa bài viết của mình.

Luồng thay thế / ngoại lệ:

1. Media tải lên lỗi, hệ thống từ chối tạo bài hoặc yêu cầu tải lại tệp.
2. Người dùng sửa hoặc xóa bài viết không thuộc quyền sở hữu, hệ thống từ chối thao tác.
3. Bài viết bị hệ thống đánh dấu vi phạm nghiêm trọng, nội dung có thể bị ẩn khỏi feed.

UC-09: Tương tác với nội dung **Mục tiêu:** Cho phép người dùng thể hiện phản hồi với nội dung trong hệ thống.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập; bài viết hoặc bình luận đích tồn tại.

Hậu điều kiện: Dữ liệu tương tác và tín hiệu hành vi được ghi nhận để phục vụ hiển thị và gợi ý nội dung.

Luồng chính:

1. Người dùng mở một bài viết hoặc bình luận.
2. Người dùng thực hiện một trong các thao tác: thích, bỏ thích, lưu, bỏ lưu, bình luận, trả lời bình luận, chia sẻ sang feed, gửi qua tin nhắn hoặc repost.
3. Hệ thống cập nhật dữ liệu tương tác tương ứng.
4. Nếu thao tác có thể ảnh hưởng gợi ý nội dung, hệ thống ghi nhận thêm sự kiện hành vi.

5. Hệ thống trả trạng thái mới và có thể phát sinh thông báo cho chủ sở hữu nội dung.

Luồng thay thế / ngoại lệ:

1. Nội dung không tồn tại hoặc đã bị xóa, hệ thống từ chối thao tác.
2. Người dùng gửi bình luận rỗng hoặc nội dung không hợp lệ, hệ thống yêu cầu chỉnh sửa.
3. Thao tác chia sẻ qua tin nhắn thất bại do hộp thoại đích không hợp lệ.

UC-10: Xem feed cá nhân hóa **Mục tiêu:** Cung cấp feed Friends và Explore phù hợp với sở thích và hành vi của người dùng.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập; hệ thống có dữ liệu bài viết công khai hoặc từ quan hệ theo dõi.

Hậu điều kiện: Danh sách bài viết được trả về cùng thông tin phân trang và các chỉ số liên quan.

Luồng chính:

1. Người dùng mở màn hình feed và chọn tab Friends hoặc Explore.
2. Hệ thống xác định nguồn dữ liệu ứng với tab đã chọn.
3. Hệ thống tính điểm hoặc sắp xếp bài viết dựa trên hồ sơ hành vi, chủ đề, mức độ phổ biến và các tín hiệu thời gian.
4. Hệ thống trả danh sách bài viết đã phân trang cho client.
5. Người dùng có thể gửi phản hồi như see more, hide hoặc not interested.
6. Hệ thống ghi nhận phản hồi để cập nhật hồ sơ gợi ý ở các lần truy xuất sau.

Luồng thay thế / ngoại lệ:

1. Hồ sơ hành vi chưa đủ dữ liệu, hệ thống fallback sang các tín hiệu mặc định như độ mới và độ phổ biến.

2. Không có bài viết phù hợp, hệ thống trả danh sách rỗng.

UC-11: Giải thích thuật ngữ văn hóa mạng **Mục tiêu:** Hỗ trợ người dùng hiểu nội dung có chứa slang hoặc tham chiếu văn hóa mạng.

Actor chính: Người dùng, Gemini API.

Tiền điều kiện: Bài viết đã được lưu hoặc đã qua quá trình phân tích thuật ngữ.

Hậu điều kiện: Thuật ngữ được phát hiện, gắn vị trí trong nội dung và có giải thích phục vụ hiển thị ở client.

Luồng chính:

1. Sau khi bài viết được tạo hoặc chỉnh sửa, hệ thống dò tìm thuật ngữ theo từ điển slang hiện hành.
2. Hệ thống gom các thuật ngữ phát hiện được và gọi Gemini API một lần để lấy giải thích theo ngữ cảnh.
3. Hệ thống ghép kết quả giải thích với vị trí xuất hiện trong nội dung và lưu vào bài viết.
4. Khi người dùng xem bài viết, client tải dữ liệu cultural terms từ backend.
5. Người dùng nhấn vào thuật ngữ được highlight để xem nghĩa, nguồn gốc và ghi chú ngữ cảnh.
6. Nếu phát hiện giải thích sai hoặc không phù hợp, người dùng có thể gửi báo lỗi thuật ngữ.

Luồng thay thế / ngoại lệ:

1. Không phát hiện thuật ngữ nào, hệ thống kết thúc pipeline mà không gọi AI.
2. Gemini API lỗi hoặc trả dữ liệu sai định dạng, hệ thống fallback về kết quả rỗng để không làm hỏng luồng tạo bài.
3. Người dùng yêu cầu phân tích lại, hệ thống xóa kết quả cũ và chạy pipeline mới.

UC-12: Nhận và quản lý thông báo **Mục tiêu:** Cho phép người dùng theo dõi các sự kiện liên quan đến tài khoản và nội dung của mình.

Actor chính: Người dùng, Pusher/Socket.IO.

Tiền điều kiện: Người dùng đã đăng nhập; thiết bị đã được đăng ký nếu dùng push notification.

Hậu điều kiện: Thiết bị được đăng ký hoặc gỡ đăng ký; trạng thái thông báo được cập nhật đúng.

Luồng chính:

1. Người dùng đăng ký token thiết bị để nhận thông báo đẩy.
2. Khi có sự kiện như tương tác, theo dõi, nhấn tin hoặc xử lý báo cáo, hệ thống tạo thông báo tương ứng.
3. Người dùng mở danh sách thông báo.
4. Hệ thống trả danh sách thông báo, số lượng chưa đọc và trạng thái đã xem.
5. Người dùng đánh dấu một thông báo đã đọc, đánh dấu tất cả đã đọc hoặc đã xem.
6. Người dùng có thể xóa từng thông báo, xóa toàn bộ thông báo đã đọc hoặc xóa toàn bộ.

Luồng thay thế / ngoại lệ:

1. Token thiết bị không hợp lệ, hệ thống từ chối đăng ký.
2. Người dùng thao tác trên thông báo không thuộc về mình, hệ thống từ chối.

UC-13: Nhắn tin và gọi thoại **Mục tiêu:** Cung cấp kênh giao tiếp thời gian thực giữa các người dùng.

Actor chính: Người dùng, Cloudinary, Pusher/Socket.IO.

Tiền điều kiện: Người dùng đã đăng nhập; tồn tại hộp thoại trực tiếp hoặc nhóm, hoặc có thể tạo mới.

Hậu điều kiện: Tin nhắn, trạng thái đọc, trạng thái trực tuyến hoặc lịch sử cuộc gọi

được cập nhật.

Luồng chính:

1. Người dùng mở danh sách cuộc trò chuyện hoặc tạo nhóm chat mới.
2. Hệ thống trả danh sách direct box, group box hoặc chi tiết nhóm.
3. Người dùng gửi tin nhắn văn bản hoặc tệp đính kèm.
4. Nếu có tệp, hệ thống nhận tệp, tải lên nơi lưu trữ phù hợp và tạo bản ghi tin nhắn.
5. Hệ thống phát tin nhắn mới qua kênh thời gian thực cho các thành viên liên quan.
6. Người dùng có thể sửa, thu hồi hoặc xóa tin nhắn theo quyền.
7. Người dùng có thể đánh dấu đã đọc, tìm kiếm trong hộp thoại, xem media đã gửi, cập nhật thông tin nhóm, thêm hoặc xóa thành viên, rời nhóm hoặc phân quyền quản trị nhóm.
8. Người dùng có thể khởi tạo cuộc gọi và cập nhật trạng thái cuộc gọi.

Luồng thay thế / ngoại lệ:

1. Người dùng không thuộc hộp thoại hoặc không có quyền quản trị nhóm, hệ thống từ chối thao tác quản lý nhóm.
2. Tệp vượt giới hạn kích thước hoặc không tải lên được, hệ thống từ chối gửi tin nhắn đính kèm.
3. Tin nhắn đã bị thu hồi hoặc xóa, hệ thống không cho chỉnh sửa tiếp.

UC-14: Gửi báo cáo vi phạm **Mục tiêu:** Cho phép người dùng chủ động phản ánh nội dung hoặc tài khoản có dấu hiệu vi phạm.

Actor chính: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập; đối tượng bị báo cáo tồn tại.

Hậu điều kiện: Báo cáo được lưu và sẵn sàng cho pipeline xử lý hoặc xem xét thủ

công.

Luồng chính:

1. Người dùng chọn báo cáo bài viết, bình luận, tài khoản hoặc thuật ngữ.
2. Hệ thống yêu cầu người dùng chọn lý do và nhập mô tả bổ sung nếu cần.
3. Người dùng xác nhận gửi báo cáo.
4. Hệ thống lưu báo cáo, liên kết với đối tượng liên quan và người gửi báo cáo.
5. Tùy loại báo cáo, hệ thống có thể tăng chỉ số report hoặc đưa đối tượng vào hàng đợi xem xét.
6. Hệ thống trả kết quả gửi báo cáo thành công.

Luồng thay thế / ngoại lệ:

1. Người dùng báo cáo trùng lặp trong điều kiện không cho phép, hệ thống từ chối hoặc gộp báo cáo.
2. Đối tượng không tồn tại, hệ thống từ chối tạo báo cáo.

UC-15: Kiểm duyệt nội dung bằng AI **Mục tiêu:** Phát hiện nội dung độc hại, spam hoặc thù ghét để giảm rủi ro trên nền tảng.

Actor chính: Hệ thống, Gemini API, Quản trị viên.

Tiền điều kiện: Bài viết hoặc bình luận đã được tạo và sẵn sàng cho pipeline kiểm duyệt.

Hậu điều kiện: Nội dung được phân loại thành duyệt ngay, gắn cờ cần xem xét hoặc tự động ẩn.

Luồng chính:

1. Khi người dùng tạo hoặc cập nhật nội dung, hệ thống khởi chạy tầng kiểm tra rule-based.
2. Hệ thống kiểm tra blocklist, mẫu hate speech và tín hiệu spam.
3. Nếu phát hiện vi phạm rõ ràng, hệ thống gắn trạng thái vi phạm ngay.

4. Nếu nội dung rơi vào vùng nghi ngờ hoặc thuộc điều kiện cần AI, hệ thống gọi Gemini API.
5. Hệ thống nhận điểm toxic, hate speech và spam từ AI, sau đó kết hợp với tín hiệu từ tầng rule-based.
6. Hệ thống ra quyết định cuối cùng: duyệt, gắn cờ human review hoặc tự động ẩn.
7. Quản trị viên có thể tiếp tục xem xét các nội dung bị gắn cờ hoặc bị báo cáo.

Luồng thay thế / ngoại lệ:

1. Gemini API lỗi hoặc timeout, hệ thống dùng cơ chế fallback để không làm thất bại thao tác tạo nội dung.
2. Nội dung bị người dùng báo cáo nhiều lần, hệ thống có thể tăng mức ưu tiên kiểm duyệt.
3. Kết quả AI không hợp lệ, hệ thống dùng điểm fallback thấp và ghi nhận lý do.

UC-16: Quản trị báo cáo và nội dung **Mục tiêu:** Hỗ trợ quản trị viên kiểm soát rủi ro nội dung và người dùng trên nền tảng.

Actor chính: Quản trị viên.

Tiền điều kiện: Quản trị viên đã đăng nhập với quyền hợp lệ.

Hậu điều kiện: Báo cáo được xử lý; bài viết, bình luận hoặc tài khoản được cập nhật trạng thái quản trị phù hợp.

Luồng chính:

1. Quản trị viên mở dashboard quản trị hoặc hàng đợi báo cáo.
2. Hệ thống trả danh sách báo cáo đang chờ, bài viết, bình luận và người dùng liên quan.
3. Quản trị viên xem chi tiết đối tượng bị báo cáo và lịch sử hoạt động liên quan.

4. Quản trị viên chọn hành động như ẩn bài, hiện lại bài, xóa bài, xử lý bình luận, khóa hoặc mở khóa người dùng, đánh dấu báo cáo đã giải quyết.
5. Hệ thống lưu kết quả xử lý và ghi nhận audit log.
6. Hệ thống trả trạng thái cập nhật cho trang quản trị.

Luồng thay thế / ngoại lệ:

1. Quản trị viên không có quyền trên thao tác cụ thể, hệ thống từ chối.
2. Đối tượng đã bị xóa hoặc xử lý trước đó, hệ thống trả trạng thái hiện tại để tránh thao tác lặp.

UC-17: Quản lý từ điển slang **Mục tiêu:** Duy trì danh mục thuật ngữ văn hóa mạng phục vụ phát hiện và giải thích trong bài viết.

Actor chính: Quản trị viên.

Tiền điều kiện: Quản trị viên đã đăng nhập.

Hậu điều kiện: Từ điển slang được cập nhật, thêm mới hoặc bật tắt đúng mục cần quản lý.

Luồng chính:

1. Quản trị viên mở màn hình quản lý từ điển slang.
2. Hệ thống trả danh sách các mục slang hiện có cùng trạng thái hoạt động.
3. Quản trị viên thêm một mục mới gồm thuật ngữ, regex pattern, nghĩa và thông tin mô tả.
4. Hệ thống kiểm tra dữ liệu đầu vào và lưu mục mới.
5. Quản trị viên có thể bật hoặc tắt một mục để điều chỉnh phạm vi phát hiện.
6. Hệ thống cập nhật từ điển và làm mới cache theo chu kỳ hệ thống.

Luồng thay thế / ngoại lệ:

1. Regex hoặc dữ liệu mục slang không hợp lệ, hệ thống từ chối lưu.
2. Quản trị viên thao tác trên mục không tồn tại, hệ thống trả lỗi phù hợp.

UC-18: Theo dõi thống kê hệ thống **Mục tiêu:** Cung cấp cho quản trị viên cái nhìn tổng quan về trạng thái vận hành và hiệu quả hệ thống.

Actor chính: Quản trị viên.

Tiền điều kiện: Quản trị viên đã đăng nhập.

Hậu điều kiện: Quản trị viên nhận được các chỉ số và dữ liệu giám sát cần thiết để ra quyết định.

Luồng chính:

1. Quản trị viên truy cập dashboard hoặc khu vực thống kê.
2. Hệ thống trả các số liệu tổng quan về người dùng, bài viết, vi phạm, báo cáo và tương tác.
3. Quản trị viên xem chỉ số hiệu năng AI, thống kê vi phạm theo ngày và thông tin audit log.
4. Khi cần, quản trị viên mở sâu vào thống kê của từng người dùng, bài viết hoặc nhóm dữ liệu liên quan.

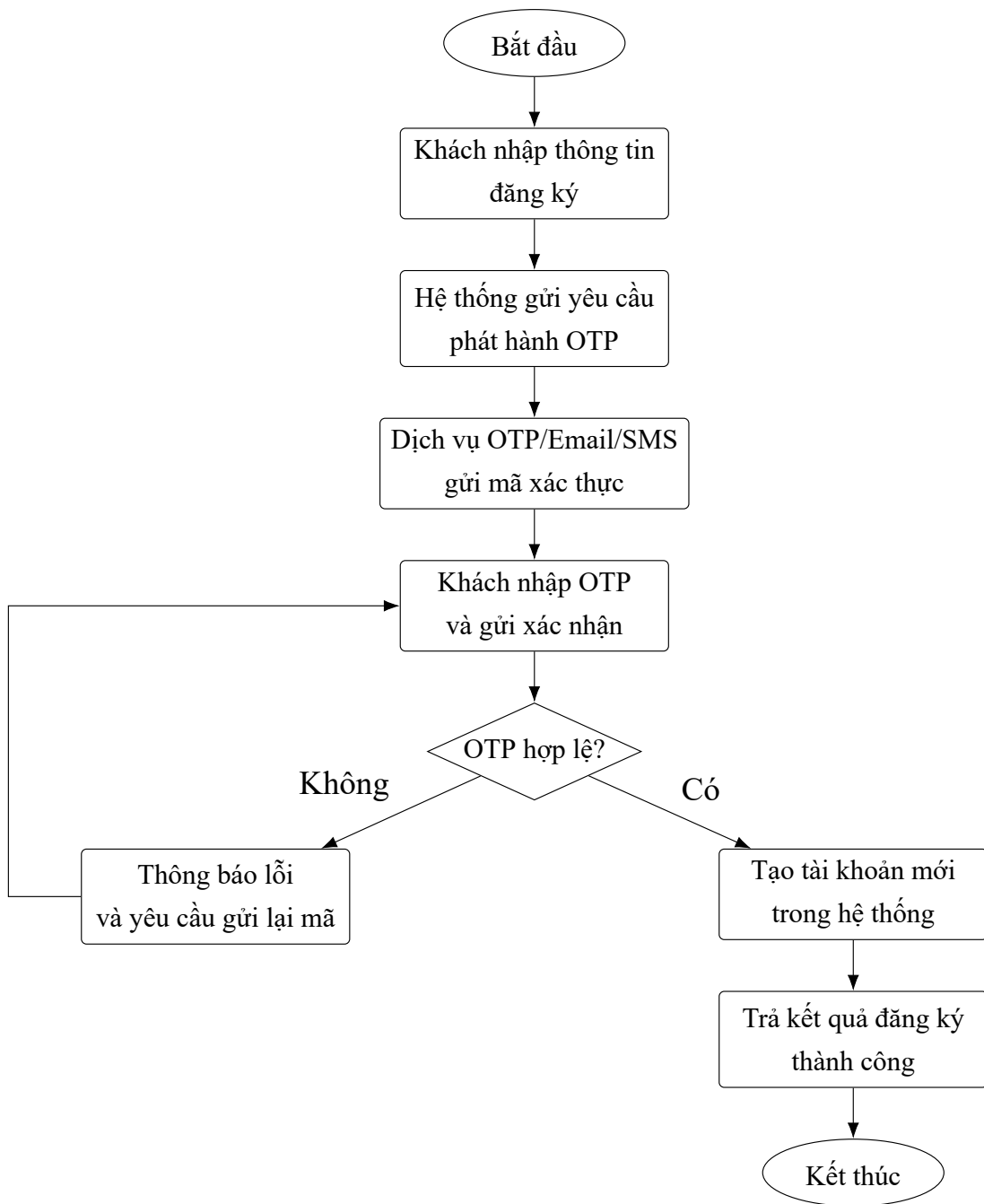
Luồng thay thế / ngoại lệ:

1. Dữ liệu thống kê tạm thời chưa sẵn sàng, hệ thống trả kết quả rỗng hoặc giá trị mặc định.
2. Người dùng không có quyền quản trị cố truy cập chức năng này, hệ thống từ chối.

3.5. Sơ đồ Activity

Phần này mô tả luồng xử lý của các chức năng nghiệp vụ quan trọng nhất trong hệ thống. Các sơ đồ activity được chọn theo tiêu chí bao quát những luồng có nhiều nhánh quyết định, tương tác với dịch vụ ngoài và tác động trực tiếp đến trải nghiệm người dùng hoặc công việc quản trị.

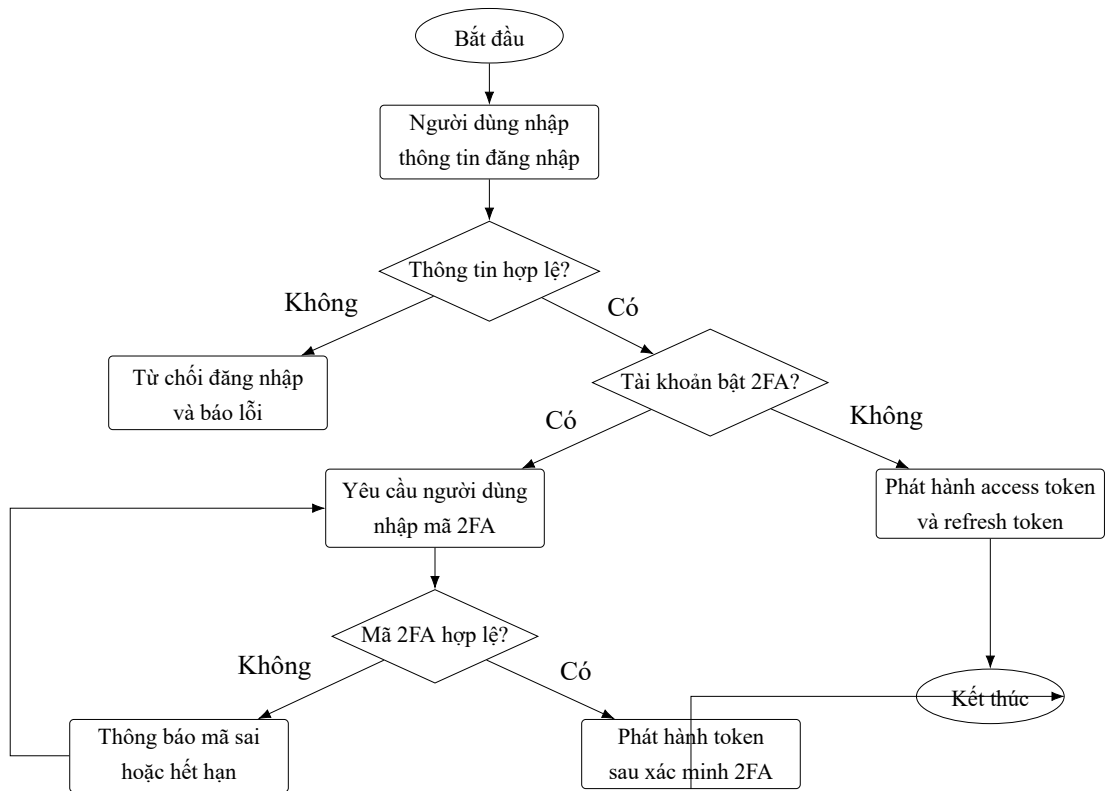
Đăng ký tài khoản và xác thực OTP



Hình 3.5: Sơ đồ activity đăng ký tài khoản và xác thực OTP

Nguồn: Nhóm tác giả

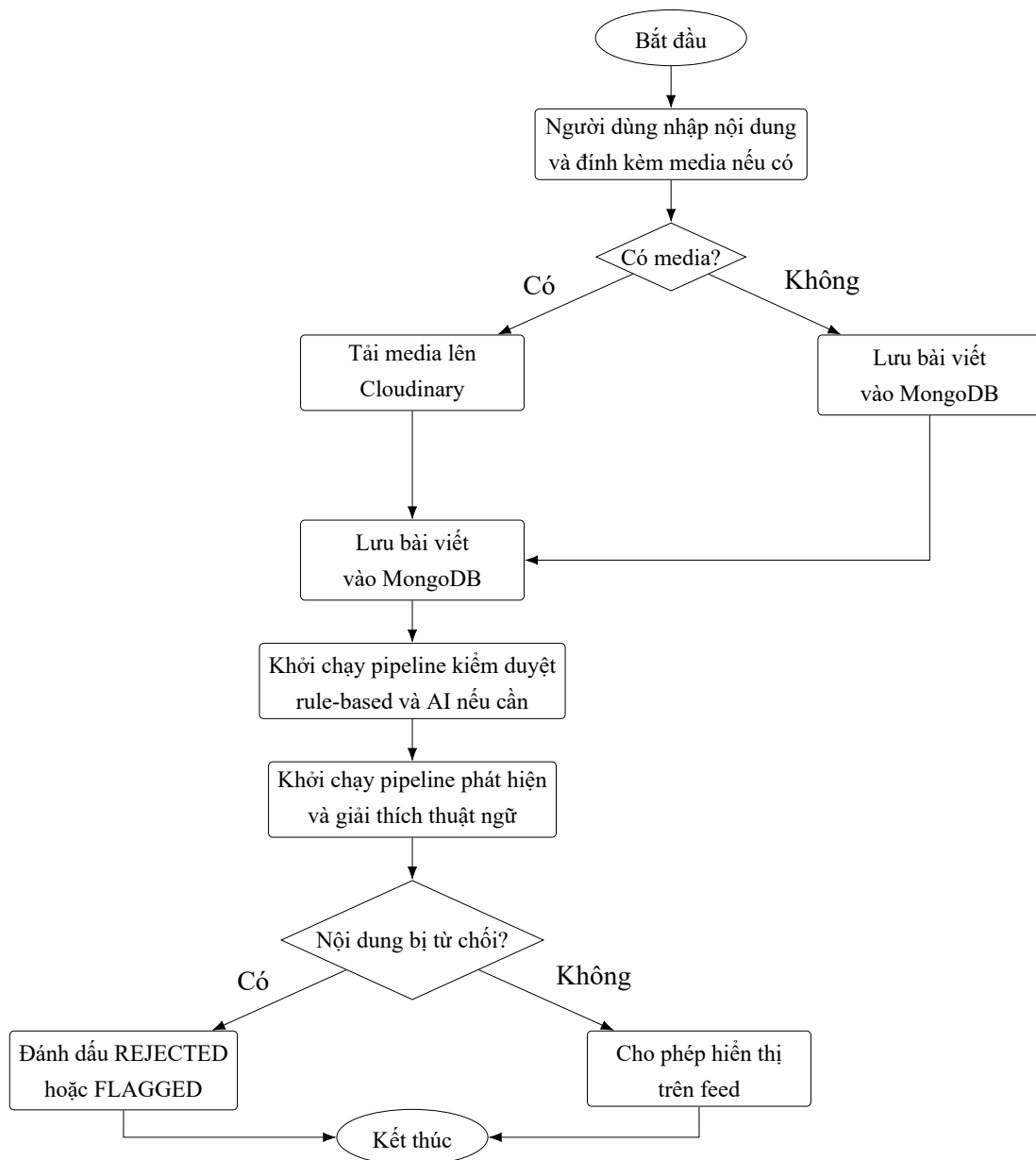
Đăng nhập và xác thực hai yếu tố



Hình 3.6: Sơ đồ activity đăng nhập và xác thực hai yếu tố

Nguồn: Nhóm tác giả

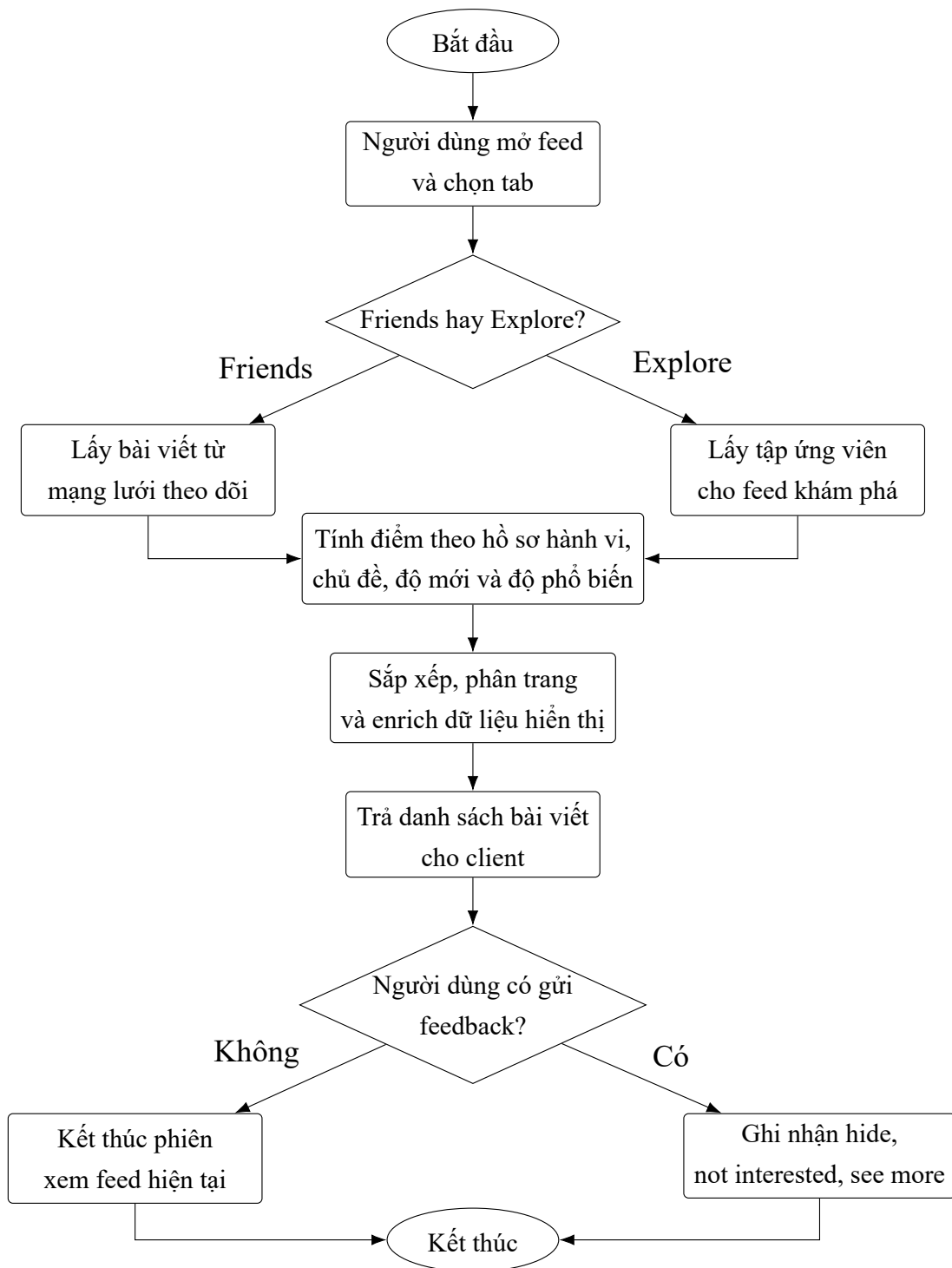
Tạo bài viết và xử lý hậu kỳ bằng AI



Hình 3.7: Sơ đồ activity tạo bài viết và xử lý hậu kỳ bằng AI

Nguồn: Nhóm tác giả

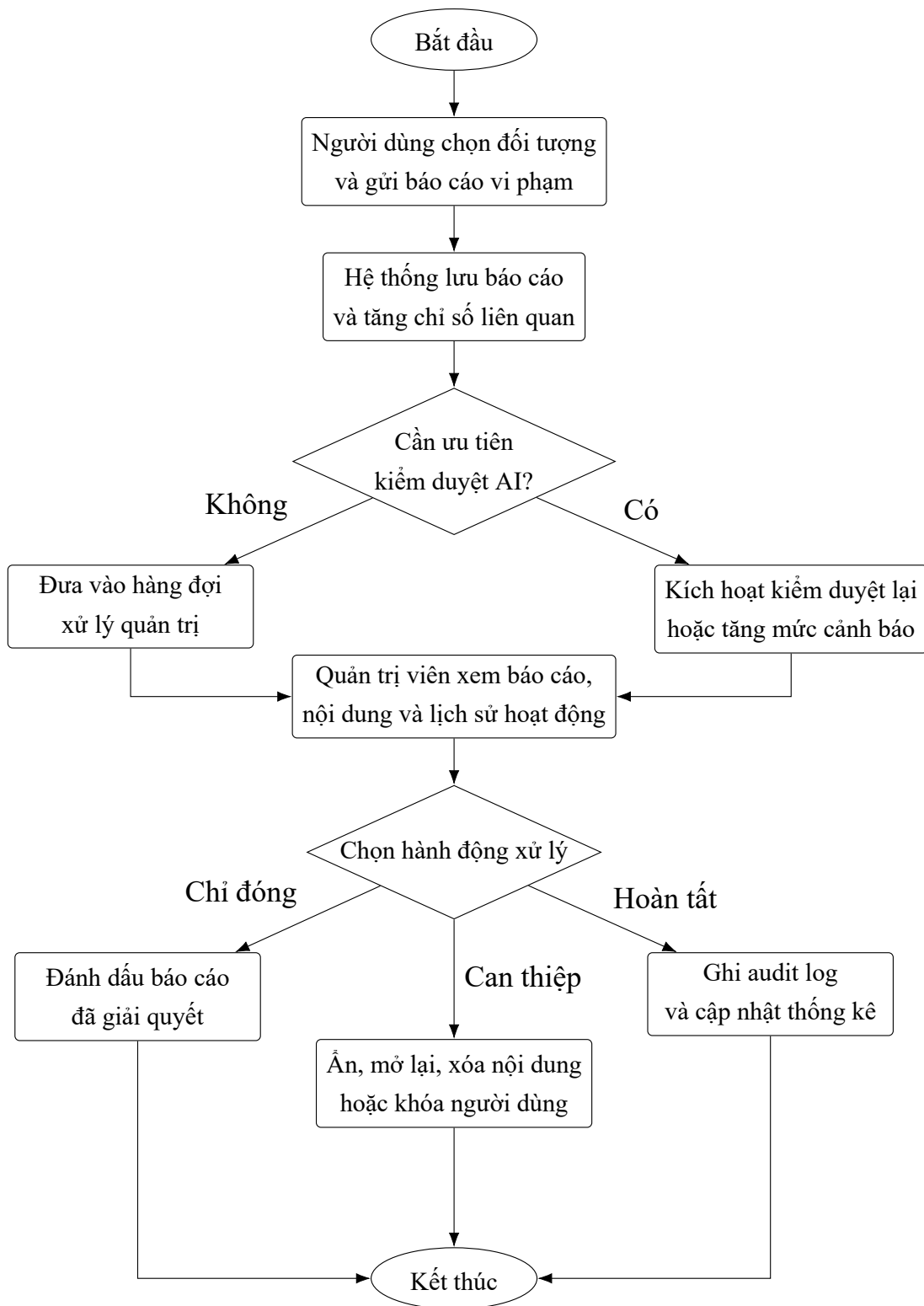
Truy xuất feed cá nhân hóa



Hình 3.8: Sơ đồ activity truy xuất feed cá nhân hóa

Nguồn: Nhóm tác giả

Báo cáo vi phạm và xử lý quản trị



Hình 3.9: Sơ đồ activity báo cáo vi phạm và xử lý quản trị

Nguồn: Nhóm tác giả

3.6. Thiết kế dữ liệu

3.6.1. Sơ đồ lớp (Class Diagram)

3.6.2. Sơ đồ ERD

3.6.3. Mô tả chi tiết các collection

Phần này sẽ được bổ sung sau khi hoàn thiện sơ đồ ERD và bảng mô tả dữ liệu.

Chương 4. XÂY DỰNG ỨNG DỤNG

Chương này trình bày giao diện của ứng dụng di động, trang quản trị và các kết quả tích hợp module hỗ trợ thông minh. Khi hoàn thiện bản Word, mỗi ảnh cần được chèn bằng chức năng **References** → **Insert Caption**, đặt nhãn “Hình”, đánh số theo chương và ghi nguồn “Nhóm tác giả” nếu ảnh do nhóm tự chụp.

4.1. Giao diện ứng dụng di động

Các hình bên dưới là những màn hình thực tế được sử dụng trong ứng dụng di động. Tên mỗi hình đều được chuẩn hóa theo mẫu “Màn hình ...” để thống nhất với phần mô tả giao diện trong chương.

Số hình	Màn hình cần chụp	Chú thích đề xuất
Hình 4.1	Màn hình Đăng nhập	Giao diện đăng nhập bằng email hoặc số điện thoại
Hình 4.2	Màn hình Đăng ký	Giao diện đăng ký tài khoản người dùng
Hình 4.3	Màn hình Xác thực tài khoản	Giao diện xác thực email hoặc số điện thoại bằng mã OTP
Hình 4.4	Màn hình Xác thực hai yếu tố	Giao diện thiết lập và xác nhận mã xác thực hai yếu tố
Hình 4.5	Màn hình Feed Khám phá	Giao diện feed Khám phá với nội dung được xếp hạng cá nhân hóa
Hình 4.6	Màn hình Tạo bài viết	Giao diện tạo bài viết gồm văn bản, hashtag, mention và quyền riêng tư
Hình 4.7	Màn hình Chi tiết bài viết	Giao diện chi tiết bài viết và các thông tin tương tác
Hình 4.8	Màn hình Tìm kiếm	Giao diện tìm kiếm người dùng, bài viết và hashtag

Số hình	Màn hình cần chụp	Chú thích đề xuất
Hình 4.9	Màn hình Hồ sơ cá nhân	Giao diện hồ sơ cá nhân và danh sách bài viết của người dùng
Hình 4.10	Màn hình Quan hệ người dùng	Giao diện danh sách người theo dõi, đang theo dõi và bạn thân
Hình 4.11	Màn hình Bài viết đã lưu	Giao diện quản lý các bài viết người dùng đã lưu
Hình 4.12	Màn hình Thông báo	Giao diện danh sách thông báo của người dùng
Hình 4.13	Màn hình Danh sách trò chuyện	Giao diện danh sách cuộc trò chuyện gần đây
Hình 4.14	Màn hình Chi tiết trò chuyện	Giao diện nhắn tin trực tiếp hoặc trò chuyện nhóm
Hình 4.15	Màn hình Báo cáo nội dung	Giao diện gửi báo cáo đối với bài viết, bình luận hoặc tài khoản
Hình 4.16	Màn hình Quên mật khẩu	Giao diện yêu cầu cấp lại quyền truy cập tài khoản
Hình 4.17	Màn hình Đổi mật khẩu	Giao diện nhập và xác nhận mật khẩu mới
Hình 4.18	Màn hình Giải thích thuật ngữ văn hóa mạng	Giao diện hiển thị giải thích theo ngữ cảnh cho các thuật ngữ văn hóa mạng

Welcome Back!

Enter your phone number and password to access your account

Phone Number *

Password *



☐ Remember me

[Forgot Password?](#)

Sign In

 Sign In with Google

Don't you have an account? [Sign Up](#)

Hình 4.1: Màn hình Đăng nhập

Nguồn: Nhóm tác giả

Sign up

create your new account

Full Name *

Phone Number *

Password *



Confirm Password *



Sign Up

 Sign Up with Google

Already have an account? [Sign in](#)

Hình 4.2: Màn hình Đăng ký

Nguồn: Nhóm tác giả

Verify account

Enter the 6-digit code sent to your phone.

SMS Code *

6 digits

[Use Email Instead](#)

Verify and Sign Up

[Gửi lại sau 57s](#)

[Edit Information](#)

Hình 4.3: Màn hình Xác thực tài khoản

Nguồn: Nhóm tác giả

20:57



← Two-Factor Authentication



2FA is enabled

Use an Authenticator app to protect future sign-ins.

Password *

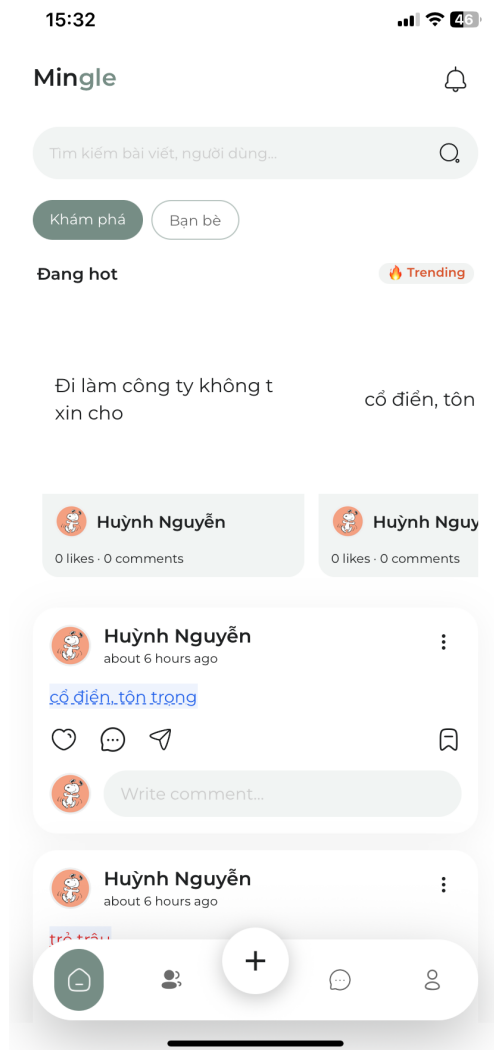


Authenticator Code *

Disable 2FA

Hình 4.4: Màn hình Xác thực hai yếu tố

Nguồn: Nhóm tác giả



Hình 4.5: Màn hình Feed Khám phá

Nguồn: Nhóm tác giả

← Tạo bài viết

 Huỳnh Nguyễn

Bạn đang nghĩ gì? Dùng #hashtag trong nội dung nếu cần.

 Add media 

 Tag friends Select friends ▼

 Location Add location ▼

Ai có thể xem?

Công khai Bạn bè Bạn thân Chỉ mình tôi

Create Post

Hình 4.6: Màn hình Tạo bài viết

Nguồn: Nhóm tác giả

20:44



← Comments



Huỳnh Nguyễn

1 day ago



[Cổ điển..tôn trọng](#)



3



3 comments



Huỳnh Nguyễn

Tôn trọng

about 1 hour ago · Like · Reply · Delete

Hide replies



Huỳnh Nguyễn · Huỳnh Nguyễn

Cổ điển

in less than a minute · Like · Reply · Delete



Huỳnh Nguyễn

Cổ điển

about 1 hour ago · Like · Reply · Delete



Write comment...

Hình 4.7: Màn hình Chi tiết bài viết

Nguồn: Nhóm tác giả

15:39



Q Enter key

Recently

Clear all

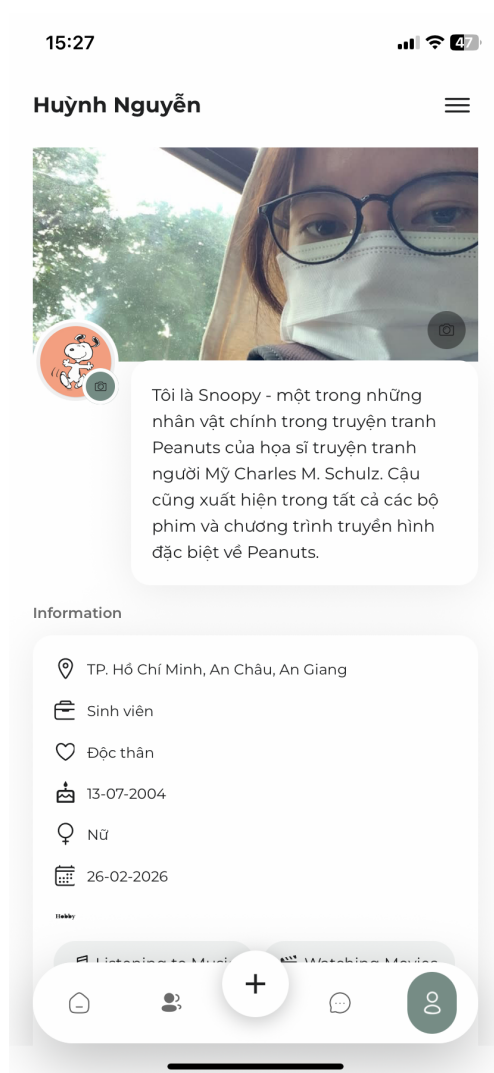


Phan Văn Đức



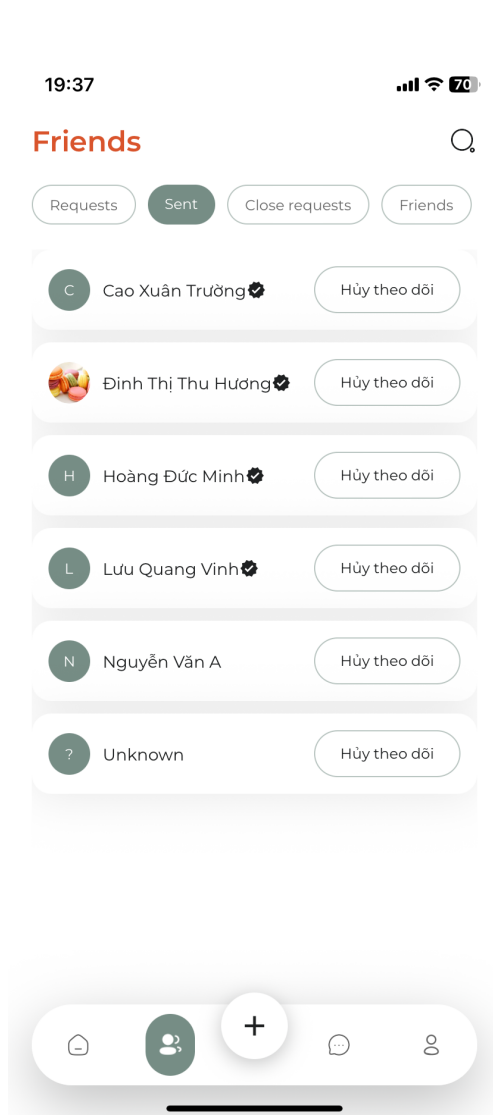
Hình 4.8: Màn hình Tìm kiếm

Nguồn: Nhóm tác giả



Hình 4.9: Màn hình Hồ sơ cá nhân

Nguồn: Nhóm tác giả



Hình 4.10: Màn hình Quan hệ người dùng

Nguồn: Nhóm tác giả

20:44

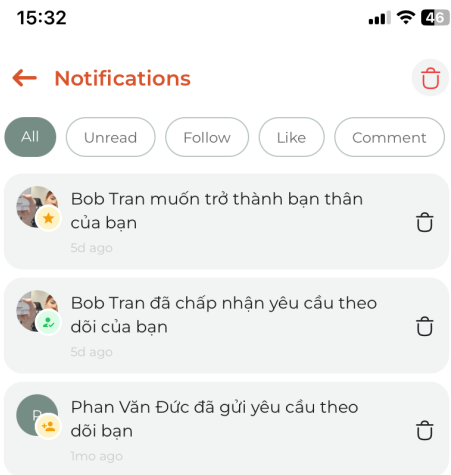


← Saved



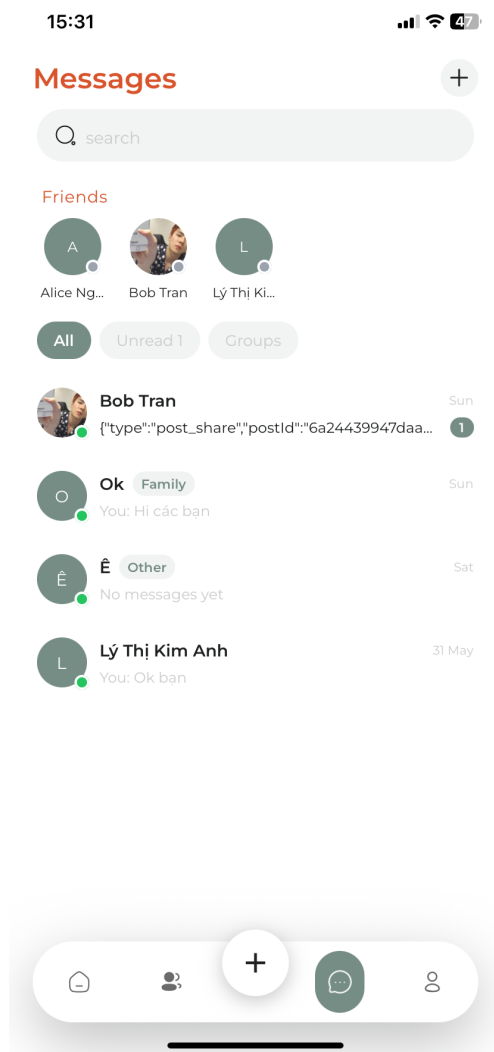
Hình 4.11: Màn hình Bài viết đã lưu

Nguồn: Nhóm tác giả



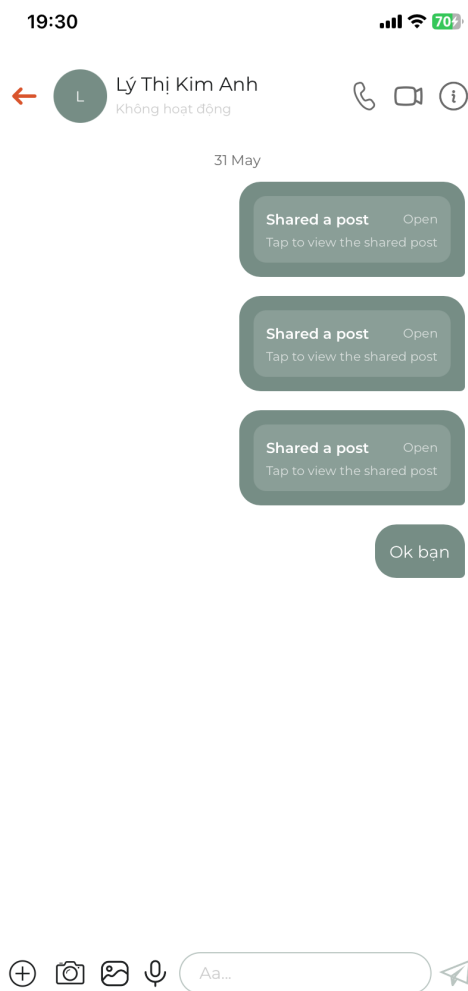
Hình 4.12: Màn hình Thông báo

Nguồn: Nhóm tác giả



Hình 4.13: Màn hình Danh sách trò chuyện

Nguồn: Nhóm tác giả



Hình 4.14: Màn hình Chi tiết trò chuyện

Nguồn: Nhóm tác giả

Báo cáo

Đóng

Hãy giúp chúng tôi hiểu vấn đề với bài viết này. Báo cáo của bạn sẽ được giữ kín.

☐ Spam / Quảng cáo

☐ Quấy rối

☐ Ngôn ngữ thù ghét

☐ Nội dung không phù hợp

☐ Lừa đảo

☐ Vi phạm bản quyền

☐ Bạo lực

☐ Thông tin sai lệch

☐ Khác

Mô tả thêm (tuỳ chọn)

Cho chúng tôi biết thêm chi tiết...

Gửi báo cáo

Hình 4.15: Màn hình Báo cáo nội dung

Nguồn: Nhóm tác giả

82

22:27



← Back

Forgot Password

Enter your phone number and we will send you a code to reset your password

Phone Number *

Send Reset Code

Hình 4.16: Màn hình Quên mật khẩu

Nguồn: Nhóm tác giả

20:45



← Change Password

Update your password to keep your account secure.

Current Password *



New Password *



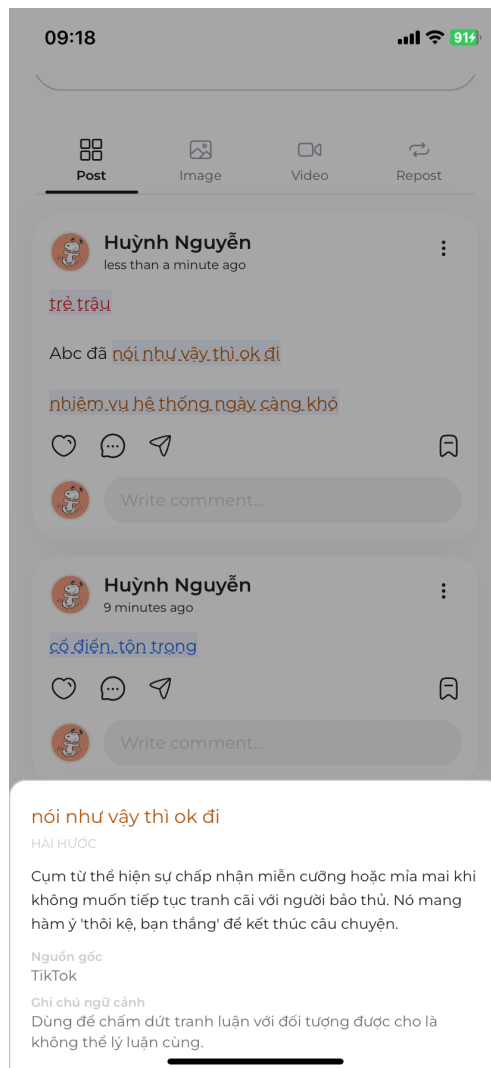
Confirm Password *



Save Password

Hình 4.17: Màn hình Đổi mật khẩu

Nguồn: Nhóm tác giả



Hình 4.18: Màn hình Giải thích thuật ngữ văn hóa mạng

Nguồn: Nhóm tác giả

4.2. Giao diện trang quản trị

Các ảnh trang quản trị nên chụp trên cùng kích thước của sổ trình duyệt, ưu tiên hiển thị đầy đủ thanh điều hướng và vùng nội dung chính.

Số hình	Màn hình cần chụp	Chú thích đề xuất
Hình 4.19	Đăng nhập quản trị	Giao diện đăng nhập dành cho quản trị viên
Hình 4.20	Dashboard	Giao diện tổng quan số liệu người dùng, nội dung và báo cáo

Số hình	Màn hình cần chụp	Chú thích đề xuất
Hình 4.21	Quản lý người dùng	Giao diện danh sách và tìm kiếm tài khoản người dùng
Hình 4.22	Chi tiết người dùng	Giao diện xem thông tin, thống kê và trạng thái tài khoản
Hình 4.23	Quản lý bài viết	Giao diện danh sách bài viết và trạng thái kiểm duyệt
Hình 4.24	Chi tiết bài viết	Giao diện xem nội dung, điểm kiểm duyệt và lịch sử xử lý bài viết
Hình 4.25	Quản lý bình luận	Giao diện danh sách và xử lý bình luận
Hình 4.26	Danh sách báo cáo	Giao diện tổng hợp các báo cáo đang chờ xử lý
Hình 4.27	Chi tiết báo cáo	Giao diện xem thông tin báo cáo và thực hiện hành động quản trị
Hình 4.28	Hiệu suất AI	Giao diện thống kê điểm và kết quả kiểm duyệt nội dung
Hình 4.29	Nhật ký hệ thống	Giao diện theo dõi audit log và hoạt động quản trị
Hình 4.30	Từ điển tiếng lóng	Giao diện quản lý từ điển thuật ngữ văn hóa mạng

4.3. Minh họa các module hỗ trợ thông minh

Nhóm hình này nên sử dụng cùng một dữ liệu mẫu trước và sau xử lý để làm rõ đầu vào, kết quả và giá trị của từng module.

Số hình	Nội dung cần chụp	Chú thích đề xuất
Hình 4.31	Kiểm duyệt nội dung	Kết quả phân loại và các điểm toxic, hate-speech, spam của một bài viết mẫu

Số hình	Nội dung cần chụp	Chú thích đề xuất
Hình 4.32	Nội dung bị gắn cờ hoặc ẩn	Trạng thái hiển thị của nội dung sau khi vượt ngưỡng kiểm duyệt
Hình 4.33	Cá nhân hóa feed	Kết quả xếp hạng feed kèm các thành phần content, popularity và social score
Hình 4.34	Giải thích thuật ngữ	Giao diện làm nổi bật thuật ngữ văn hóa mạng và hiển thị phân giải thích theo ngữ cảnh

4.4. Liên kết mã nguồn

Phần này cần cung cấp liên kết tới repository backend, ứng dụng di động và trang quản trị; đồng thời ghi rõ commit hoặc phiên bản được sử dụng để xây dựng bản báo cáo. Không đưa khóa API, mật khẩu hoặc tập tin môi trường vào repository công khai.

TÀI LIỆU THAM KHẢO

- [1] DataReportal & Manochi, "Global Social Media Statistics," Apr. 2026. [Online]. Available: <https://datareportal.com/social-media-users>
- [2] Statista, DataReportal, and We Are Social, "Number of internet and social media users worldwide," Oct. 2025. [Online]. Available: <https://www.statista.com/statistics/617136/digital-population-worldwide/>
- [3] EMARKETER, "Meta's content moderation pullback cuts takedown errors, removals," May 2025. [Online]. Available: <https://www.emarketer.com/content/meta-content-moderation-pullback-cuts-takedown-errors-removals>
- [4] Statista, "Meta's Hate Speech Problem — Hate speech on social media," 2024. [Online]. Available: <https://www.statista.com/chart/amp/21704/hate-speech-content-removed-by-facebook>
- [5] GLAAD, "Make Meta Safe: New Report Finds Increase in Harmful Content," June 2025. [Online]. Available: <https://glaad.org/make-meta-safe-new-report/>
- [6] Center for Democracy and Technology, "Intersectional Disparities within Automated Hate-speech Detection," 2025. [Online]. Available: <https://cdt.org/insights/intersectional-disparities-within-automated-hate-speech-detection/>
- [7] Springer Nature, "LLM synthetic generation to enhance online content moderation generalization," *Journal of Supercomputing*, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s00607-025-01518-8>
- [8] MDPI Social Sciences, "Trap of Social Media Algorithms: A Systematic Review on Filter Bubbles, Echo Chambers, and Their Impact on Youth," *Social Sciences*, vol. 14, no. 11, p. 301, 2025. [Online]. Available: <https://www.mdpi.com/2075-4698/15/11/301>

[9] OurMental.Health, "How TikTok's Algorithm Creates Filter Bubbles and Amplifies Biases," Dec. 2024. [Online]. Available: <https://www.ourmental.health/screen-time-sanity/how-tiktoks-algorithm-shapes-user-perceptions>

[10] Morningstar Sustainalytics, "Meta's Content Moderation Overhaul: Key Risk Considerations," 2025. [Online]. Available: [sustainalytics.com – Meta's Content Moderation Overhaul](https://www.sustainalytics.com/meta-content-moderation-overhaul)